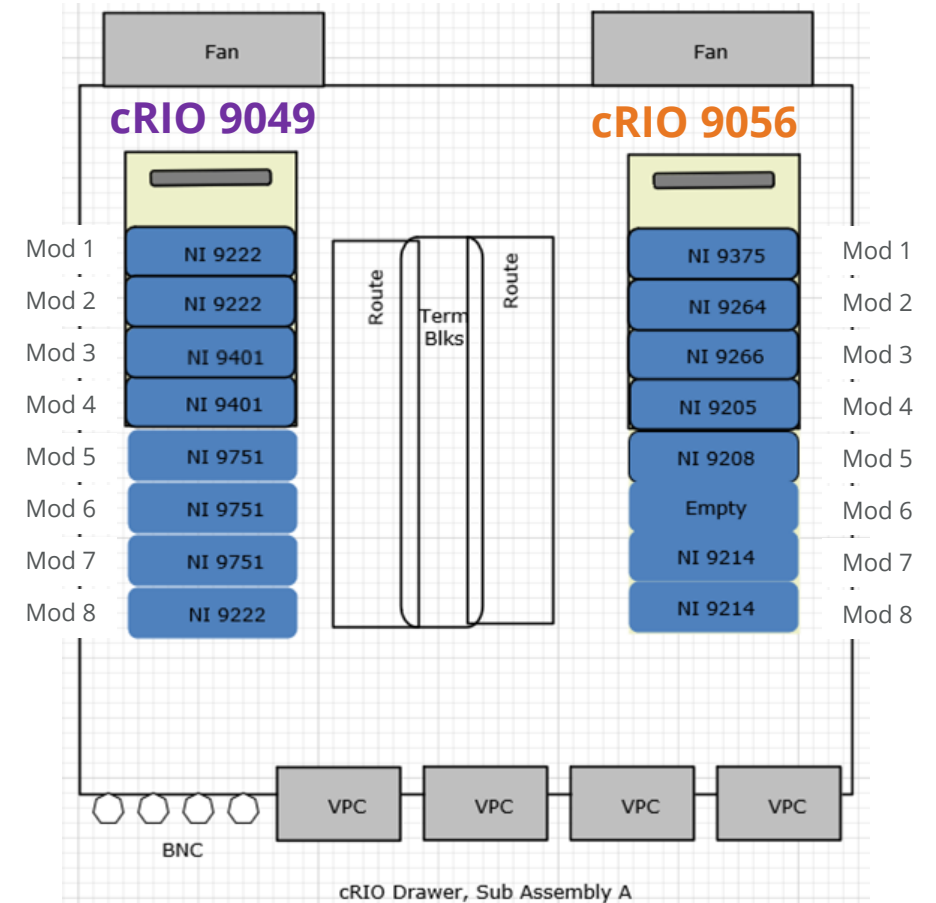
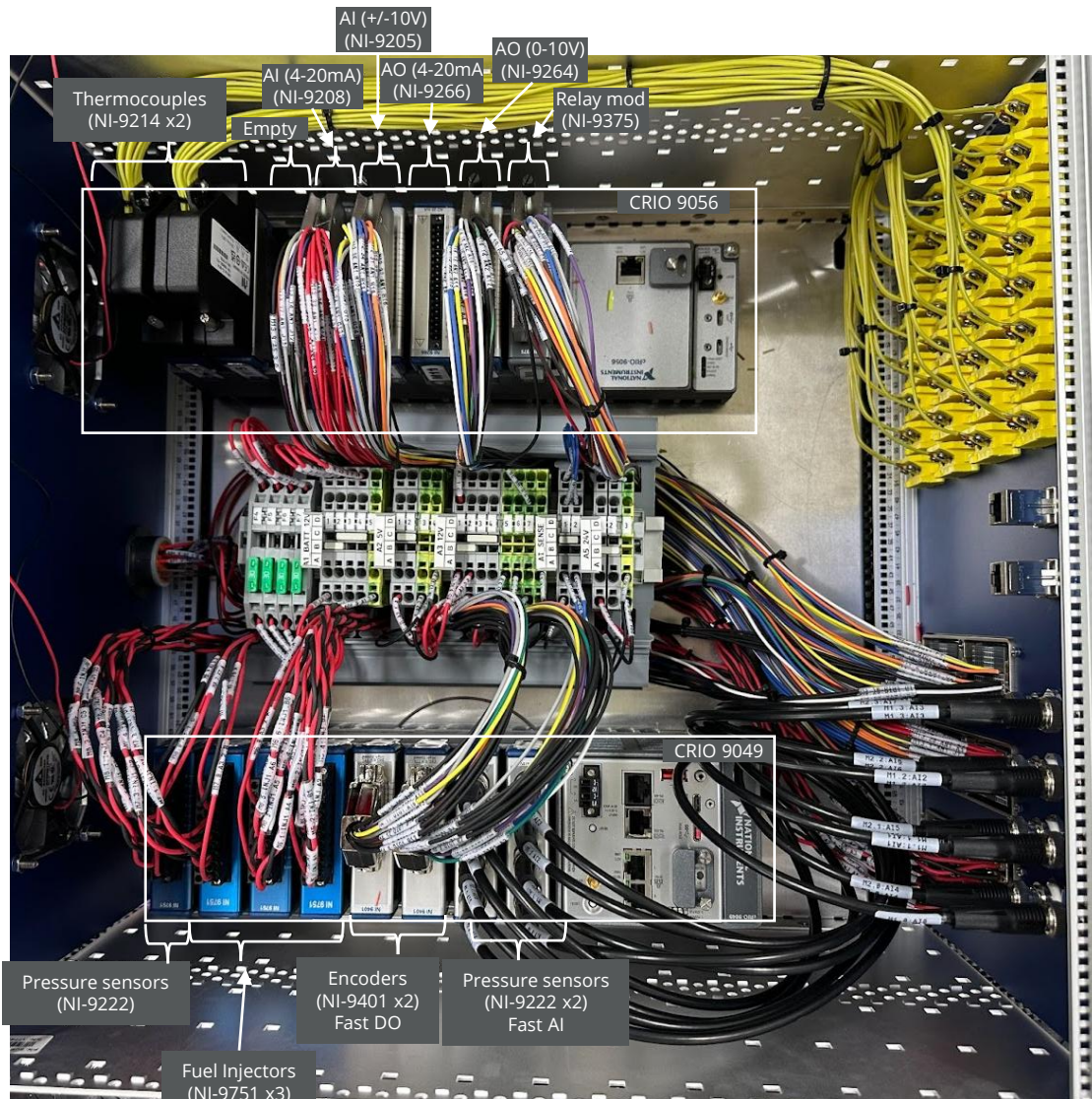


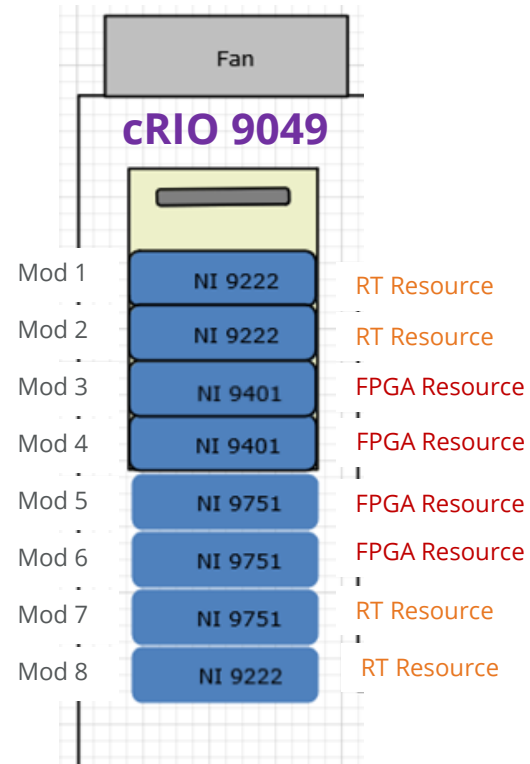
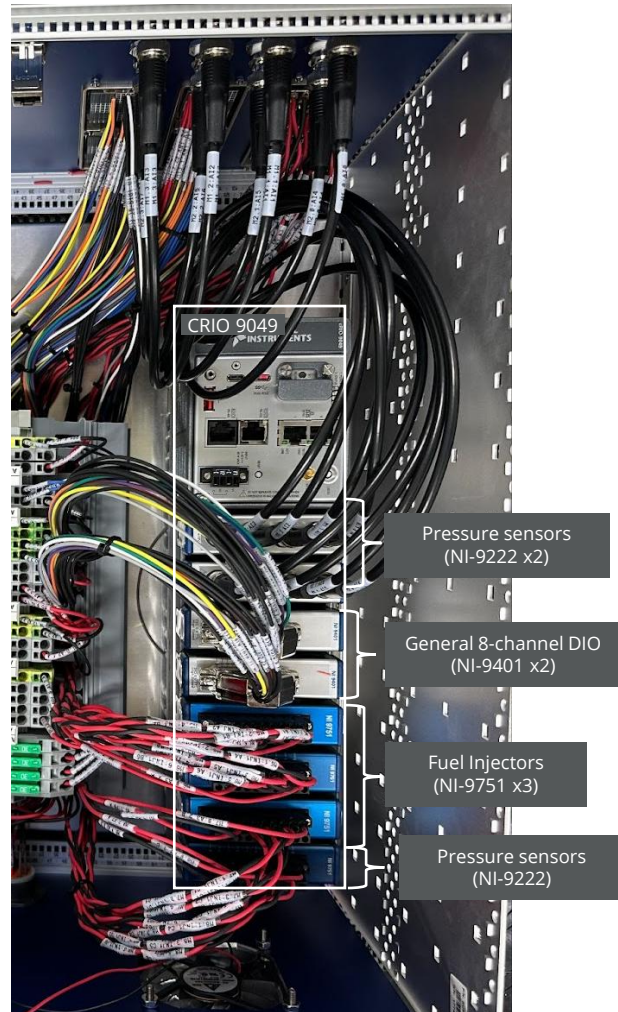
cRIO 9049

Hardware and Software

CONTROL BOX



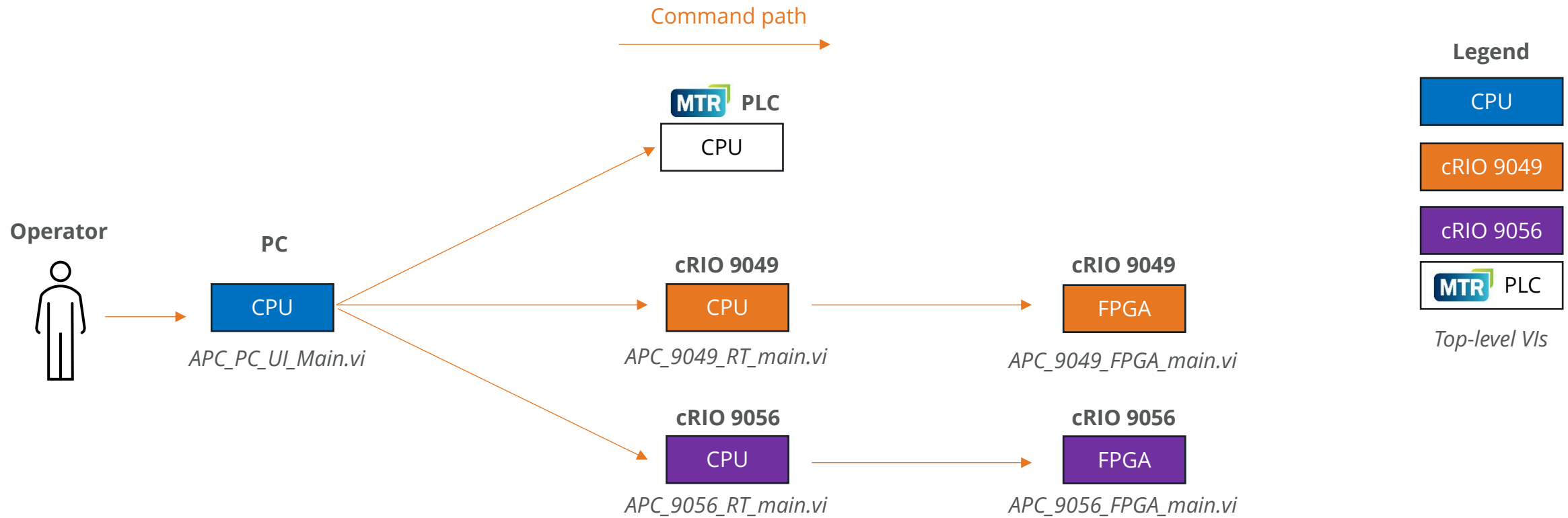
cRIO 9049 HARDWARE



Module 4: Channels DO0:5 for spark ignition

NI-9401: General-purpose 8-channel bidirectional digital I/O module. Each channel supports 5 V TTL logic and can be configured as: a) 8 digital inputs, b) 8 digital outputs, or c) 4 inputs & 4 outputs.

APC CONTROL SYSTEM SOFTWARE STRUCTURE



TOP-LEVEL cRIO 9049 RUNTIME VI

APC_9049_RT_main.vi

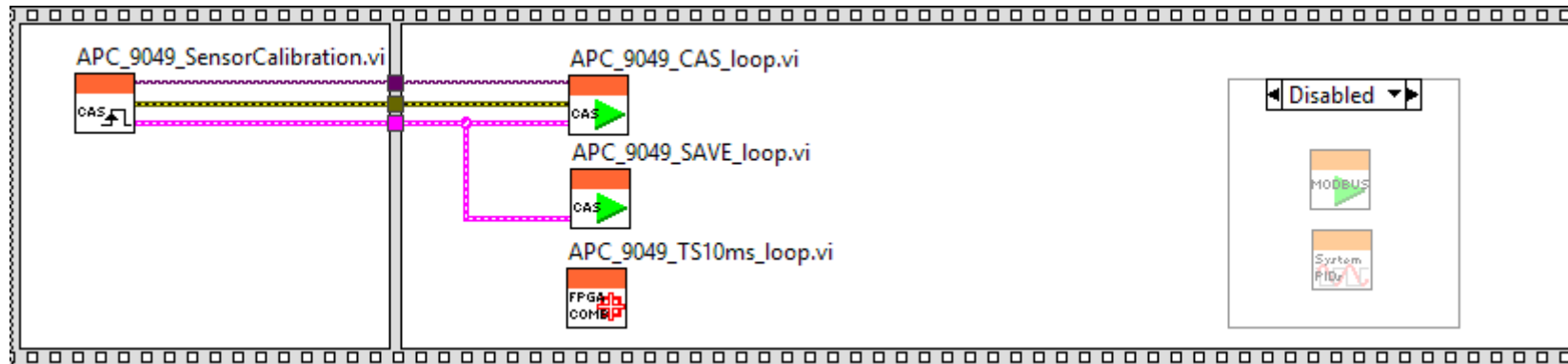


APC_9049_RT_MAIN.VI - DESCRIPTION



Description: Top-level VI for the cRIO-9049. Responsible for:

- Loading calibration parameters,
- Initializing and launching both the Crank-Angle Synchronous (CAS) and Time-Synchronous (TS) control loops,
- Managing test data acquisition and saving results to disk.



TIME-SYNCHRONOUS (TS) LOOP FOR cRIO 9049 FPGA EXECUTION CONTROL

APC_9049_TS10ms_loop.vi



REAL-TIME 10MS LOOP - cRIO 9049 - DESCRIPTION



Objective

Configure the **ignition** and **injection** settings for the engine.

REAL-TIME 10MS LOOP - cRIO 9049 - BD OVERVIEW



Opens the FPGA VI
Reference to enable
FPGA I/O

Compute engine speed

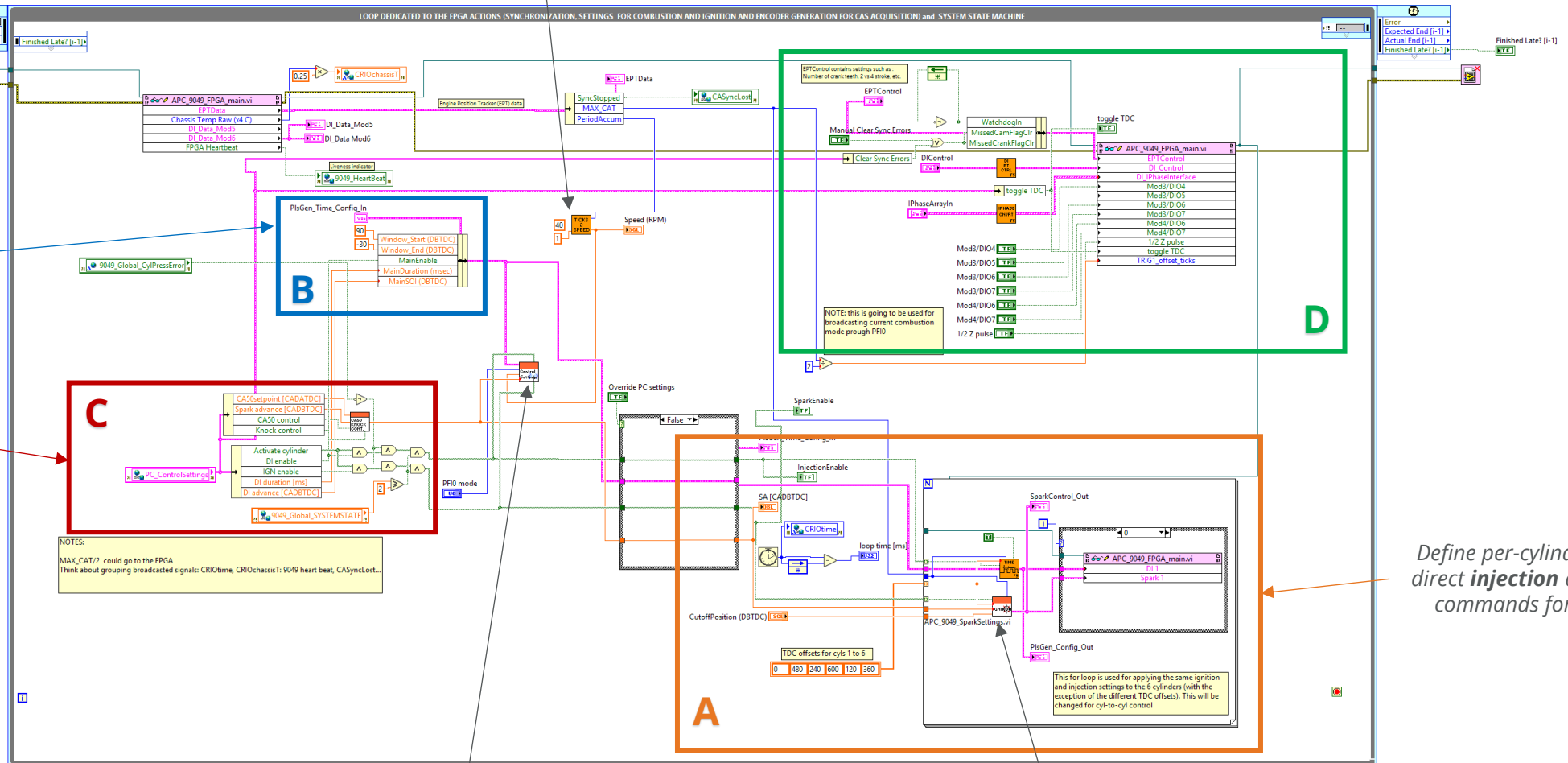
Populate the
PIGen_Time_Config_In
cluster with **direct
injection settings**.

Configure Ignition and
Injection **Enable** States

Define per-cylinder low-level
direct **injection** and **ignition**
commands for the FPGA

Share Data with PC and cRIO 9056
(APC_9049_ControlSettingsRaster.vi)

Convert spark and injection
timing to FPGA Hardware ticks
(APC_9049_SparkSettings.vi)

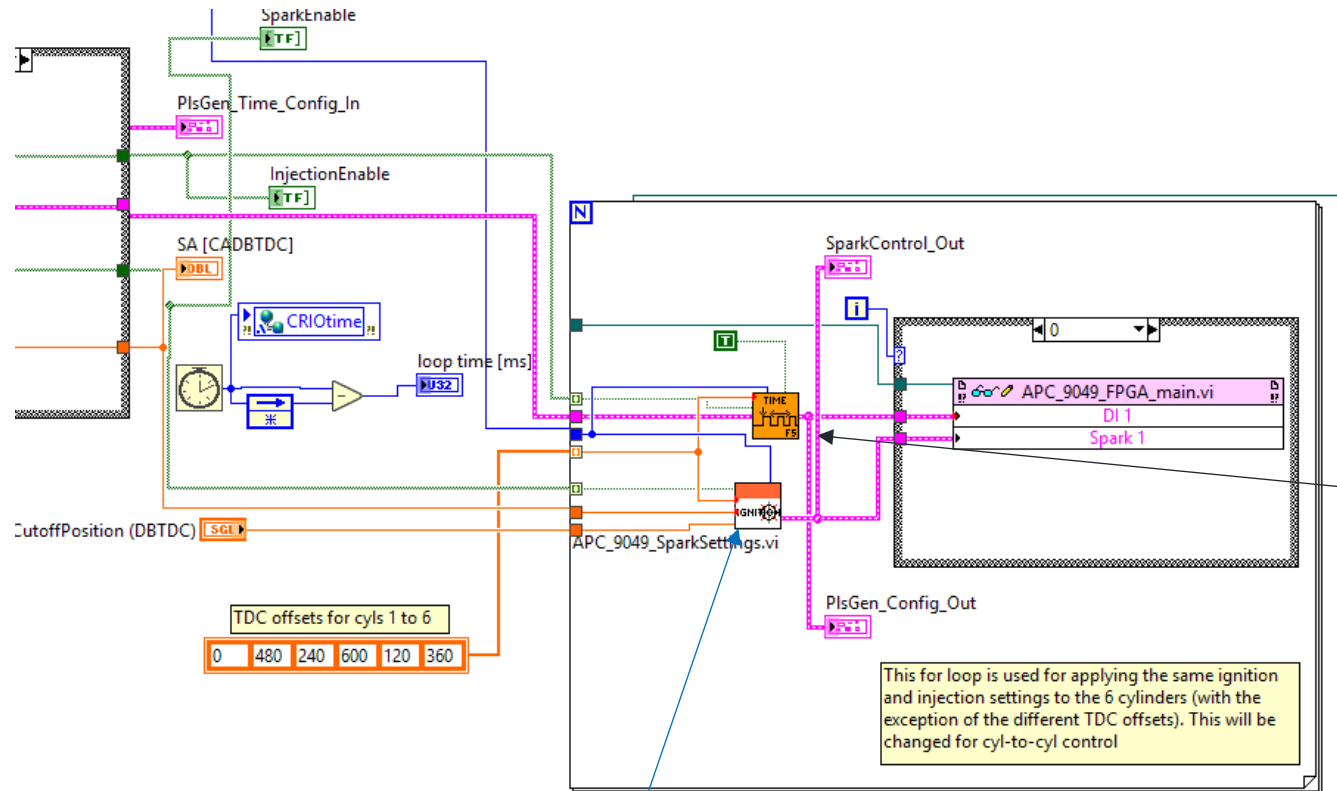


REAL-TIME 10MS LOOP - cRIO 9049 – BD DETAILS

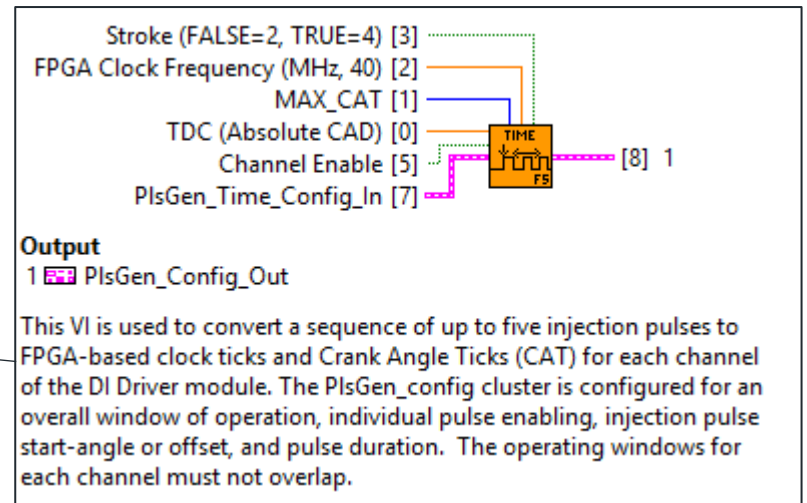
1/4



Section **A**: Define per-cylinder low-level direct **injection** and **ignition** commands for the FPGA.



PlsGen_Time_Convert_rev5.vi



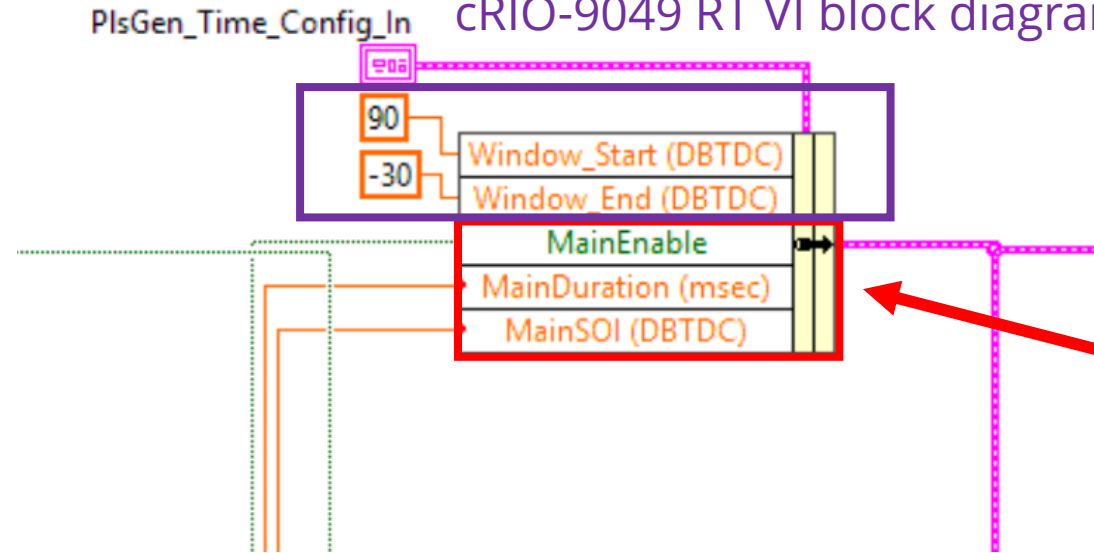
APC_9049_SparkSettings:
converts human-unit spark
timing and dwell to
hardware ticks.

REAL-TIME 10MS LOOP - cRIO 9049 – BD DETAILS 2/4

B

Section **B**: Populate the *PIGen_Time_Config_In* cluster with **direct injection settings**.

The DI window can only be configured manually within the cRIO-9049 RT VI block diagram.



These 3 DI settings can only be configured through the front panel of the Main UI.

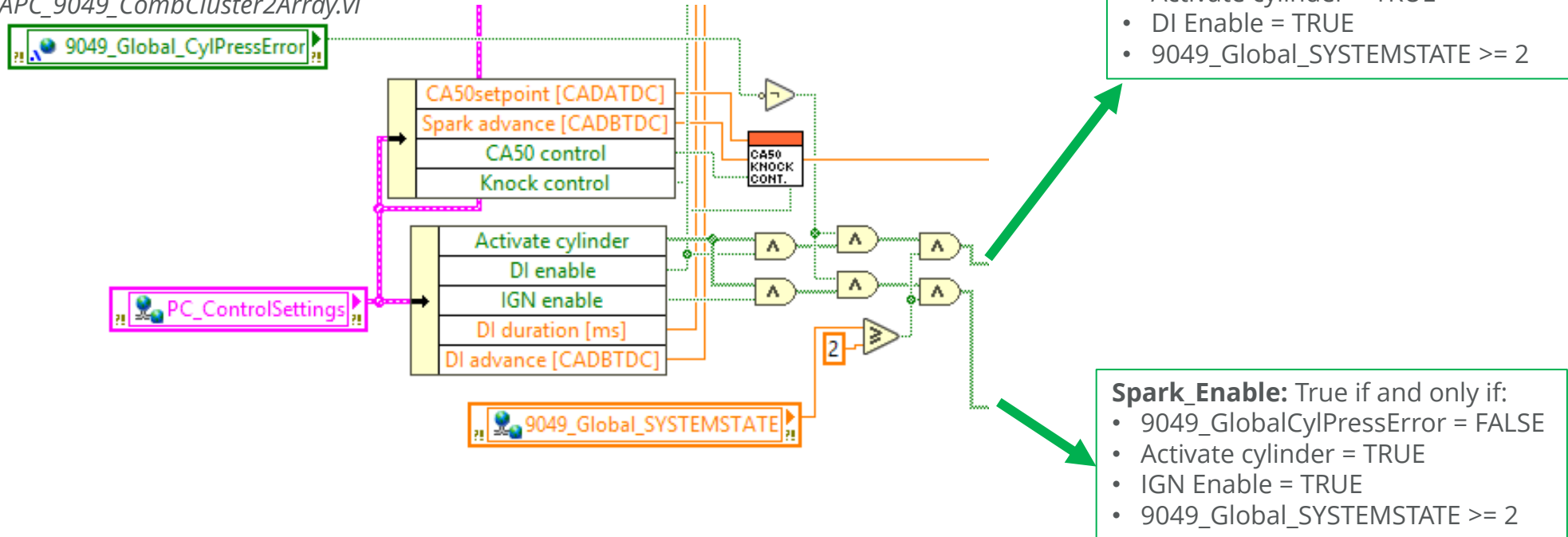


REAL-TIME 10MS LOOP - cRIO 9049 – BD DETAILS 3/4

C

Section **C**: Configure Ignition and Injection **Enable** States

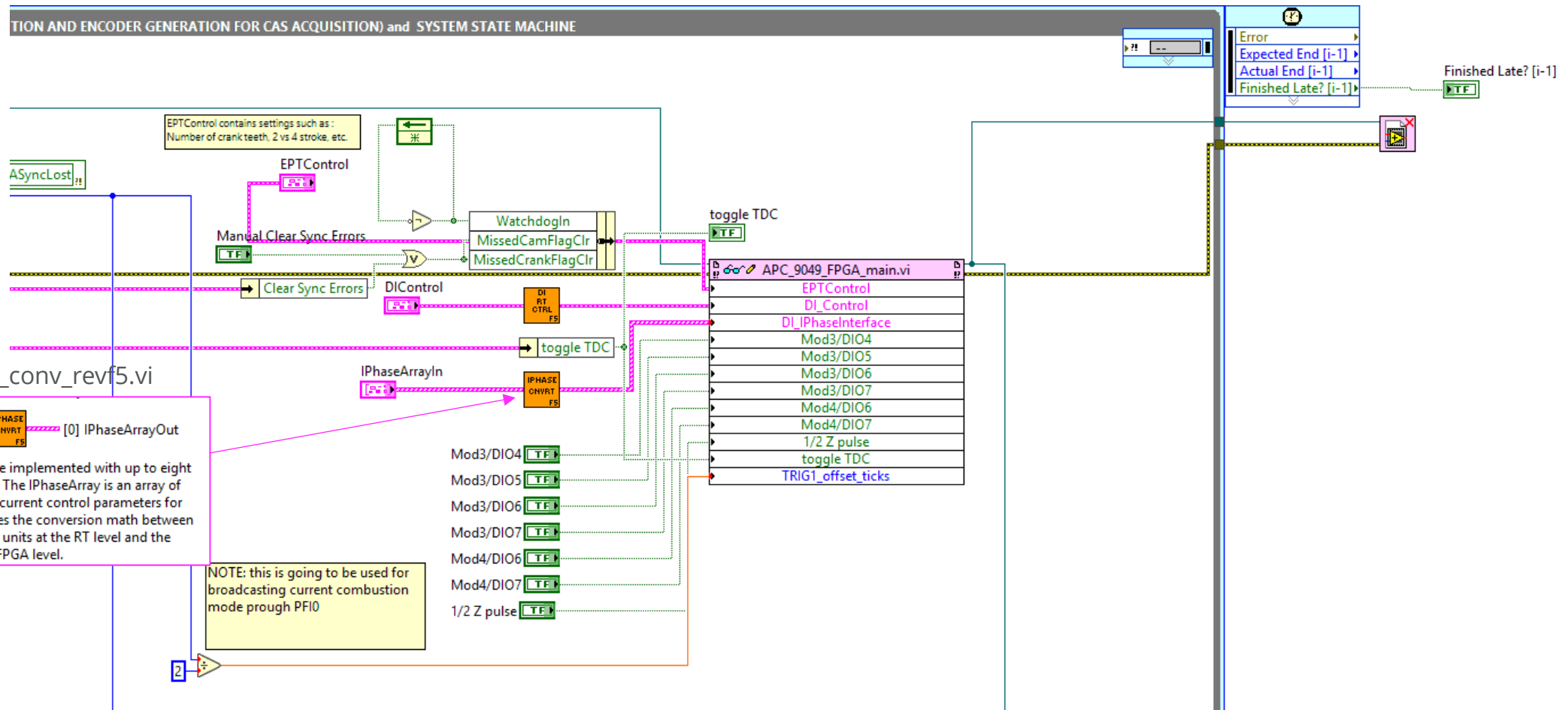
Set in APC_9049_CombCluster2Array.vi



REAL-TIME 10MS LOOP - cRIO 9049 – BD DETAILS 4/4

D

Section D: Sending Commands to the FPGA Main VI



CONVERT SPARK AND DWELL TIMING TO FPGA HARDWARE TICKS

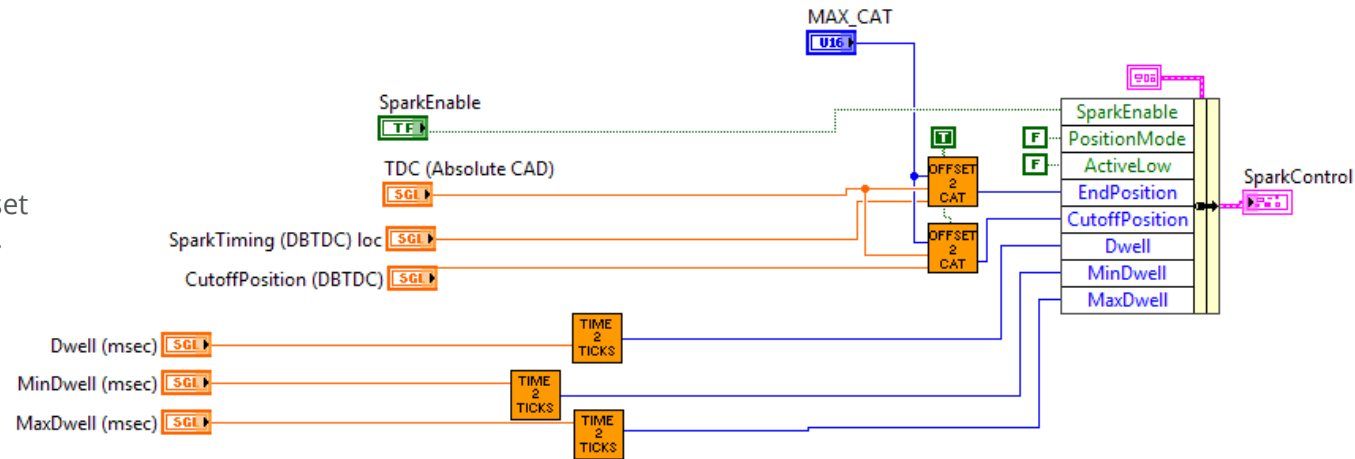
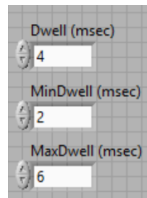
APC_9049_SparkSettings.vi

APC_9049_SPARKSETTINGS.VI - DESCRIPTION

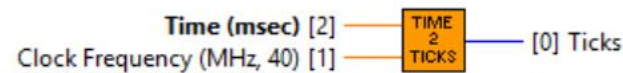
Objective

This VI converts human-unit spark and dwell timing (degrees BTDC and milliseconds) to hardware ticks tied to the crank-angle counter.

Dwell values are set locally in this VI.

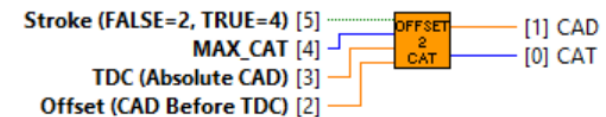


time2ticks.vi

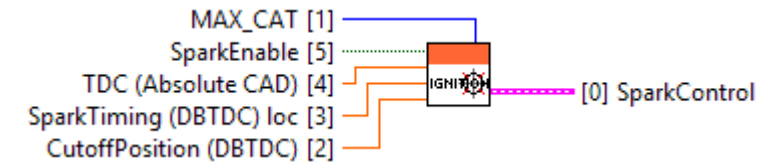


Converts time in milliseconds to clock ticks, depending on the Clock Frequency where the result will be used.

Offset2CAT.vi



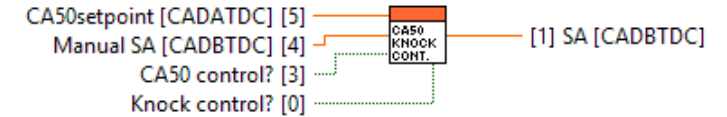
Converts an offset angle with respect to an absolute TDC to an absolute angle in Crank Angle Degrees (CAD) and Crank Angle Ticks (CAT). The Result in CAT can be wired directly to an FPGA-based fuel or spark control sub VI parameter for scheduling commands.



SET SPARK TIMING CA

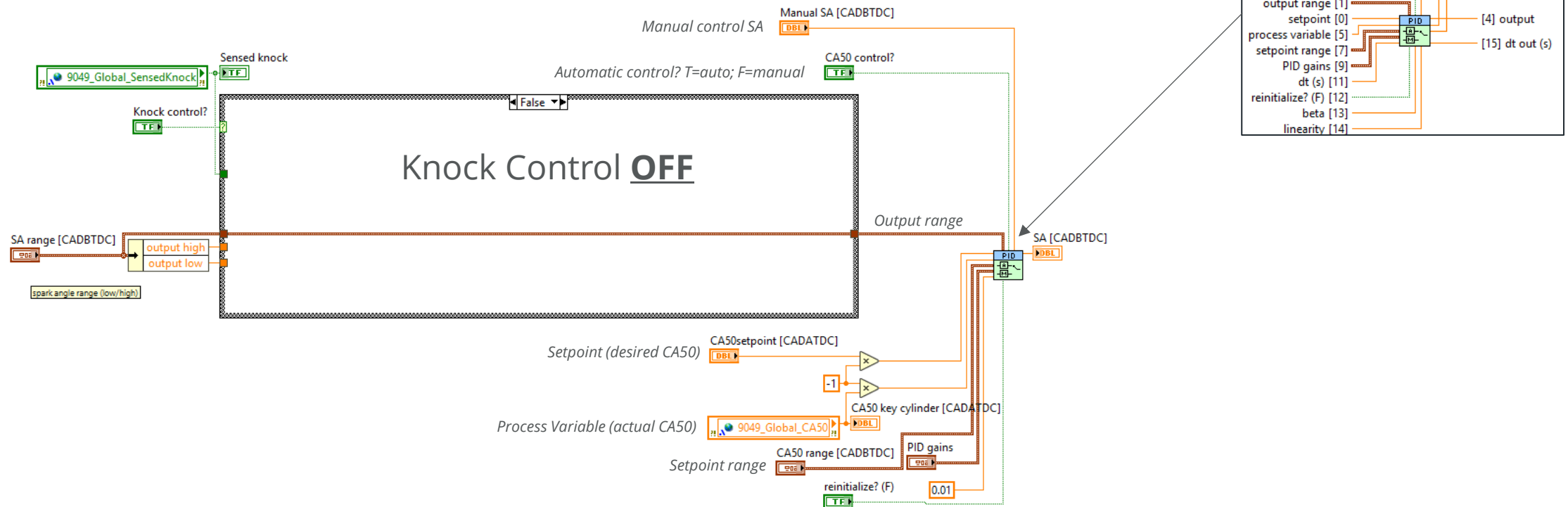
APC_9049_KnockCA50Control.vi

APC_9049_KnockCA50CONTROL.VI - DESCRIPTION

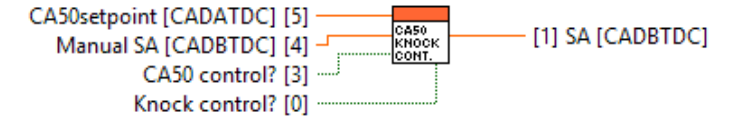


Objective

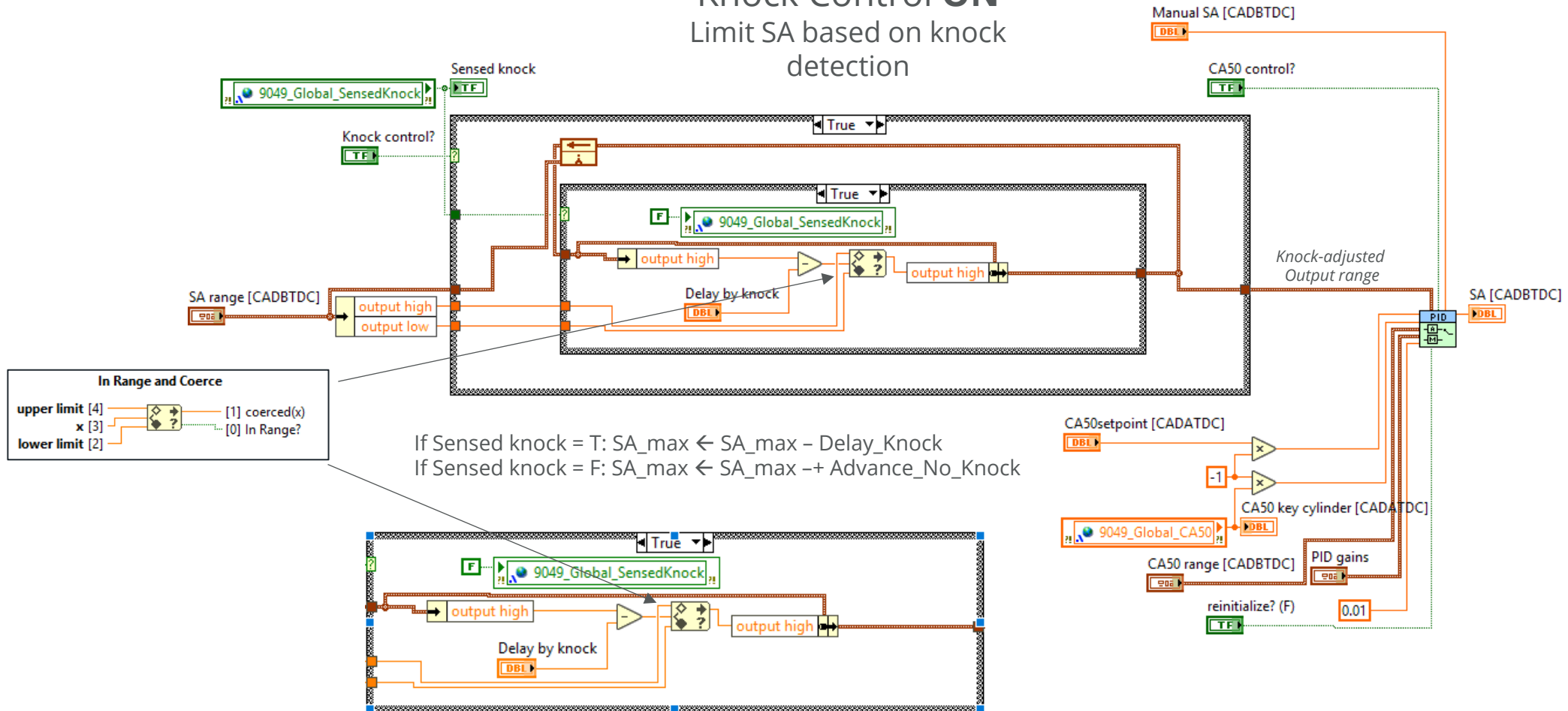
This VI is designed to control the Spark Crank Angle based on the desired CA50 value and knock detection feedback. *PID_Advanced.vi*



APC_9049_KnockCA50CONTROL.VI - DESCRIPTION



Knock Control **ON** Limit SA based on knock detection



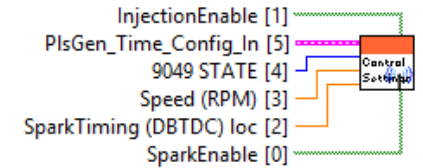
APC_9049_KNOCKCA50CONTROL.VI – INPUTS / CONTROL PARAMETERS

Parameter	Value
Knock control?	False
CA50 control?	False
CA50setpoint [CADBTDc]	N/A
Manual SA [CADBTDc]	Set by PC_ControlSettings

BUNDLE & SHARE DATA WITH PC AND cRIO 9056

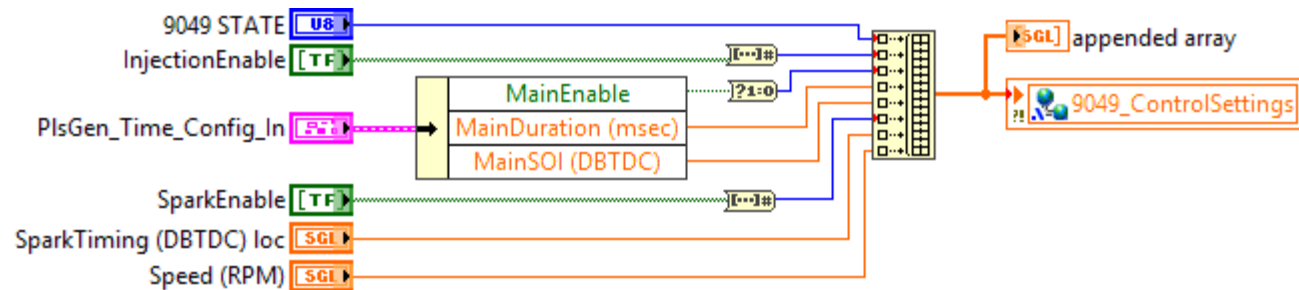
APC_9049_ControlSettingsRaster.vi

APC_9049_CONTROLSETTINGSRASTER.VI – DESCRIPTION



Objective

This VI writes data to the shared variable **9049_ControlSettings**, enabling the cRIO 9049 to transmit information to both the PC and the cRIO 9056.

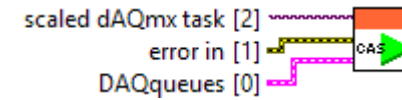


CRANK-ANGLE SYNCHRONOUS (CAS) LOOP FOR cRIO 9049 FPGA EXECUTION CONTROL

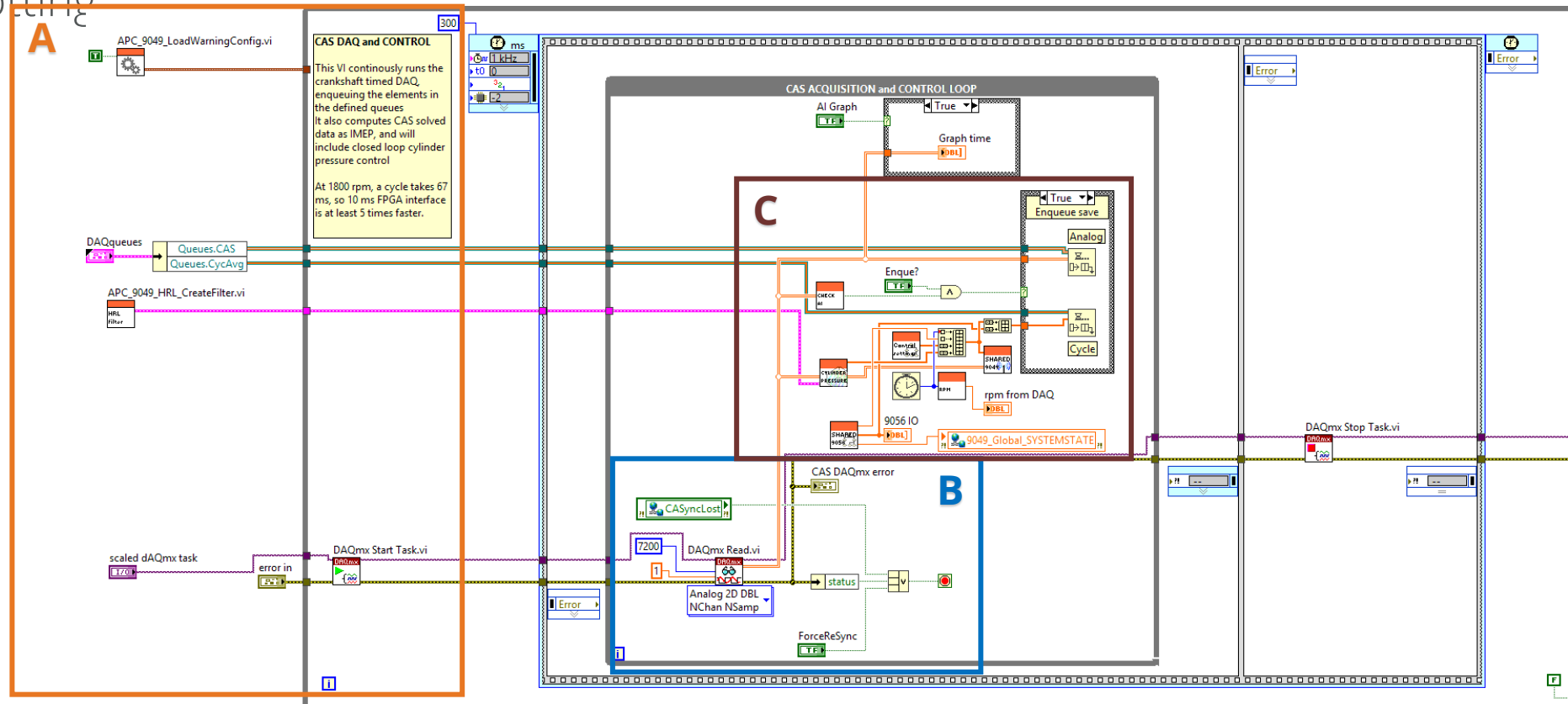
APC_9049_CAS_loop.vi



APC_9049_CAS_LOOP.VI - DESCRIPTION



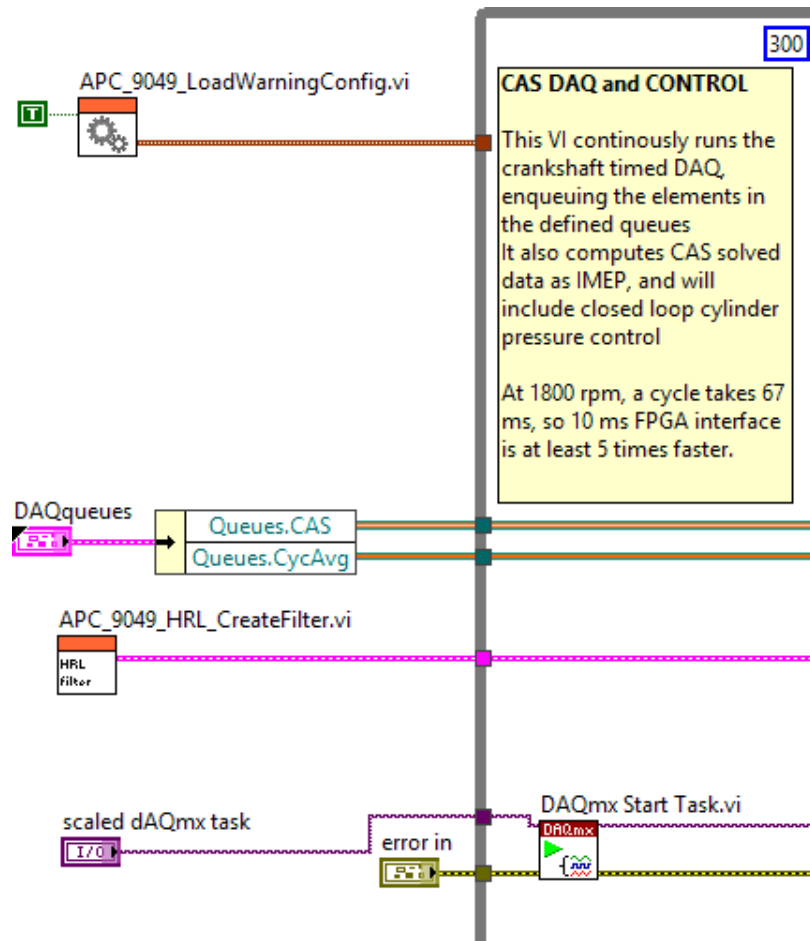
Objective: This VI runs the crankshaft-synchronous (CAS) DAQ continuously, computes CAS-derived signals (incl. IMEP inputs), and provides optional closed-loop cylinder-pressure control hooks. It also publishes data to queues for logging/processing and offers basic live plotting



APC_9049_CAS_LOOP.VI – BD DETAILS 1/X



Section A: ...



Channels and Tasks in NI-DAQmx

Virtual channels and tasks are fundamental components of NI-DAQmx.

Virtual channels, or sometimes referred to generically as channels, are software entities that encapsulate the physical channel along with other channel specific information—range, terminal configuration, and custom scaling—that formats the data. Tasks are collections of one or more virtual channels with timing, triggering, and other properties.

Tasks in NI-DAQmx

A task is a collection of one or more virtual channels with timing, triggering, and other properties. Conceptually represents a measurement or generation you want to perform. All channels in a task must be of the same I/O type as analog input or counter output. However, a task can include channels of different measurement types, such as analog input temperature channel and an analog input voltage channel. For most devices, only one task per subdevice can run at once, but some devices can run multiple tasks simultaneously. With some devices, you can include channels from multiple devices in a task. To perform a measurement or a generation with a task, follow these steps:

1. Create or load a task. You can create tasks interactively with the DAQ Assistant or programmatically in your application, such as LabVIEW or LabWindows/CVI.
2. Configure the channel, timing, and triggering properties as necessary.
3. Optionally, perform various task state transitions to prepare the task to perform the specified operation.
4. Read or write samples.
5. Clear the task.

If appropriate for your application, repeat steps 2 through 4. For instance, after reading or writing samples, you can reconfigure the virtual channel, timing, or triggering properties and then read or write additional samples based on the new configuration.

If properties need to be set to values other than their defaults for your task to be successful, your program must set those properties every time it executes. For example, if you run a program that sets property A to a nondefault value, and then run a second program that does not set property A, the second program uses the default value of property A. The only way to avoid setting properties programmatically each time a program runs is to use virtual channels and tasks created in the DAQ Assistant.

Parent topic: [Channels and Tasks in NI-DAQmx](#)

DAQmx Start Task.vi

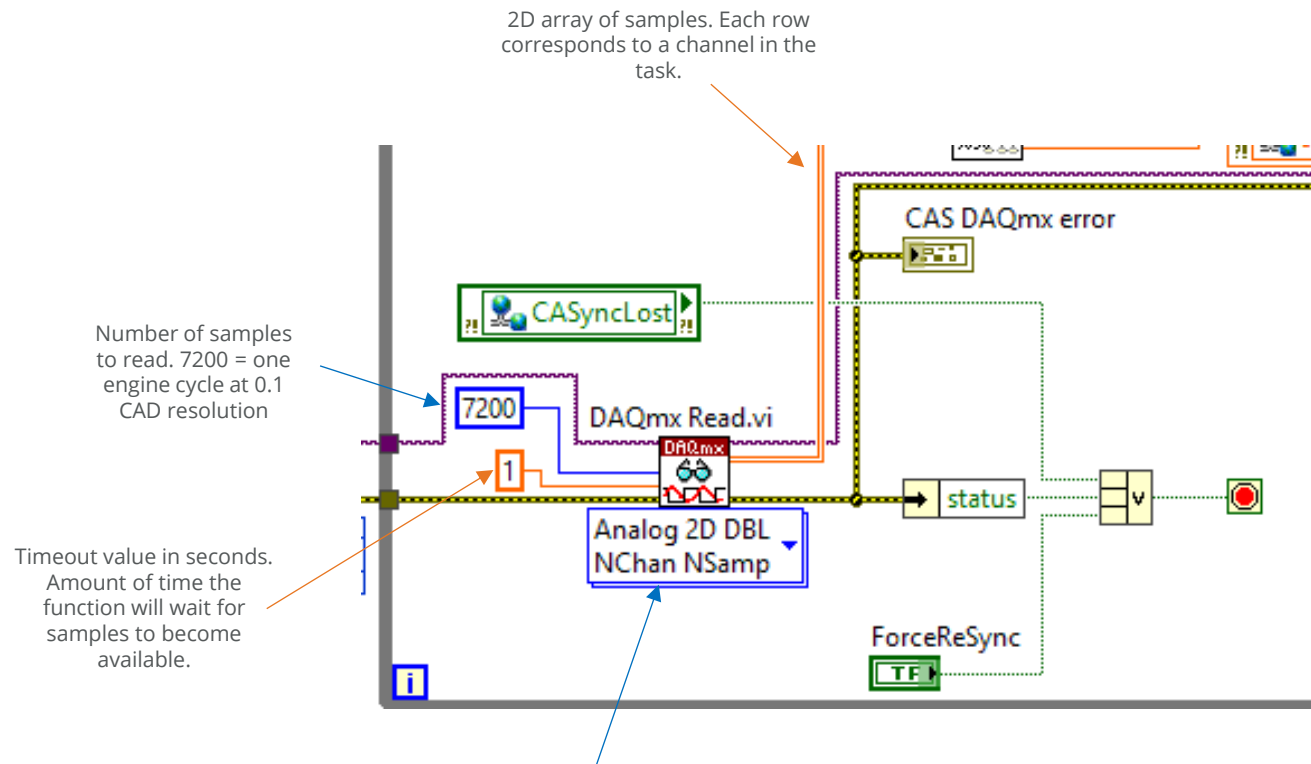
Transitions the task to the running state to begin measurement.



APC_9049_CAS_LOOP.VI – BD DETAILS 2/X

B

Section B: ...



task/channels in [0]
number of samples per channel [7]
timeout [9]
error in [11]

DAQmx

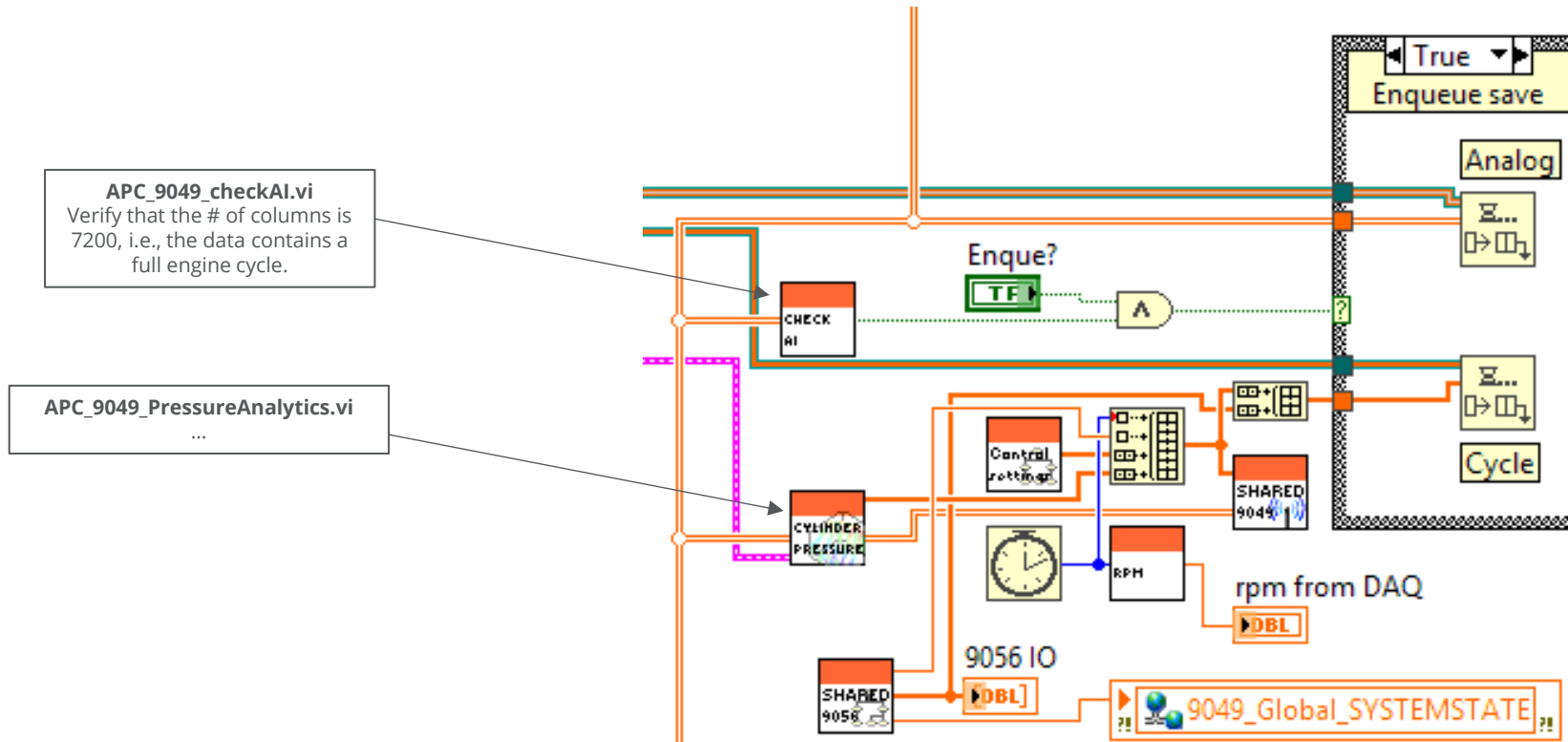
[4] task out
[6] data
[15] error out

Reads samples from a task that contains analog input channels.

APC_9049_CAS_LOOP.VI – BD DETAILS 3/X

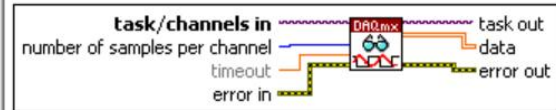


Section **C**: ...



Analog 2D DBL NChan NSamp

Reads one or more floating-point samples from a task that contains one or more [analog input channels](#).



task/channels in is the name of the [task](#) or a list of [virtual channels](#) to which the operation applies. If you provide a list of virtual channels, NI-DAQmx creates a task automatically.

number of samples per channel specifies the number of samples to read. If you leave this input unwired or set it to -1, NI-DAQmx determines how many samples to read based on if the task acquires samples continuously or acquires a finite number of samples.

If the task acquires samples continuously and you set this input to -1, this VI reads all the samples currently available in the buffer.

If the task acquires a finite number of samples and you set this input to -1, the VI waits for the task to acquire all requested samples, then reads those samples. If you set the [Read All Available Samples](#) property to TRUE, the VI reads the samples currently available in the buffer and does not wait for the task to acquire all requested samples.

timeout specifies the amount of time in seconds to wait for samples to become available. If the time elapses, the VI returns an error and any samples read before the timeout elapsed. The default timeout is 10 seconds. If you set **timeout** to -1, the VI waits indefinitely. If you set **timeout** to 0, the VI tries once to read the requested samples and returns an error if it is unable to.

error in describes error conditions that occur before this VI or function runs. The default is `no error`. If an error occurred before this VI or function runs, the VI or function passes the **error in** value to **error out**. If an error occurs while this VI or function runs, the VI or function runs normally and sets its own error status in **error out**. Use the [Simple Error Handler](#) or [General Error Handler](#) VIs to display the description of the error code. Use **error in** and **error out** to check errors and to specify execution order by wiring **error out** from one node to **error in** of the next node.

status is TRUE (X) if an error occurred before this VI or function ran or FALSE (checkmark) to indicate a warning or that no error occurred before this VI or function ran. The default is FALSE.

code is the error or warning code. The default is 0. If **status** is TRUE, **code** is a negative error code. If **status** is FALSE, **code** is 0 or a warning code.

source identifies where an error occurred. The source string includes the name of the VI that produced the error, what inputs are in error, and how to eliminate the error.

task out is a reference to the task after this VI or function runs. If you wired a channel or list of channels to **task/channels in**, NI-DAQmx creates this task automatically.

data returns a 2D array of samples. Each row corresponds to a channel in the task. Each column corresponds to a sample from each channel. The order of the channels in the array corresponds to the order in which you add the channels to the task or to the order of the channels you specify with the [Channels to Read](#) property.

NI-DAQmx scales the data to the units of the measurement, including any custom scaling you apply to the channels. Use the [DAQmx Create Virtual Channel](#) VI or the [DAQ Assistant](#) to specify these units.

error out contains error information. If **error in** indicates that an error occurred before this VI or function ran, **error out** contains the same error information. Otherwise, **error out** describes the error status that this VI or function produces. Right-click the **error out** indicator on the front panel and select **Explain Error** from the shortcut menu for more information about the error.

status is TRUE (X) if an error occurred or FALSE (checkmark) to indicate a warning or that no error occurred.

code is the error or warning code. If **status** is TRUE, **code** is a nonzero error code. If **status** is FALSE, **code** is 0 or a warning code.

source identifies where and why an error occurred. The source string includes the name of the VI that produced the error, what inputs are in error, and how to eliminate the error.

TITLE

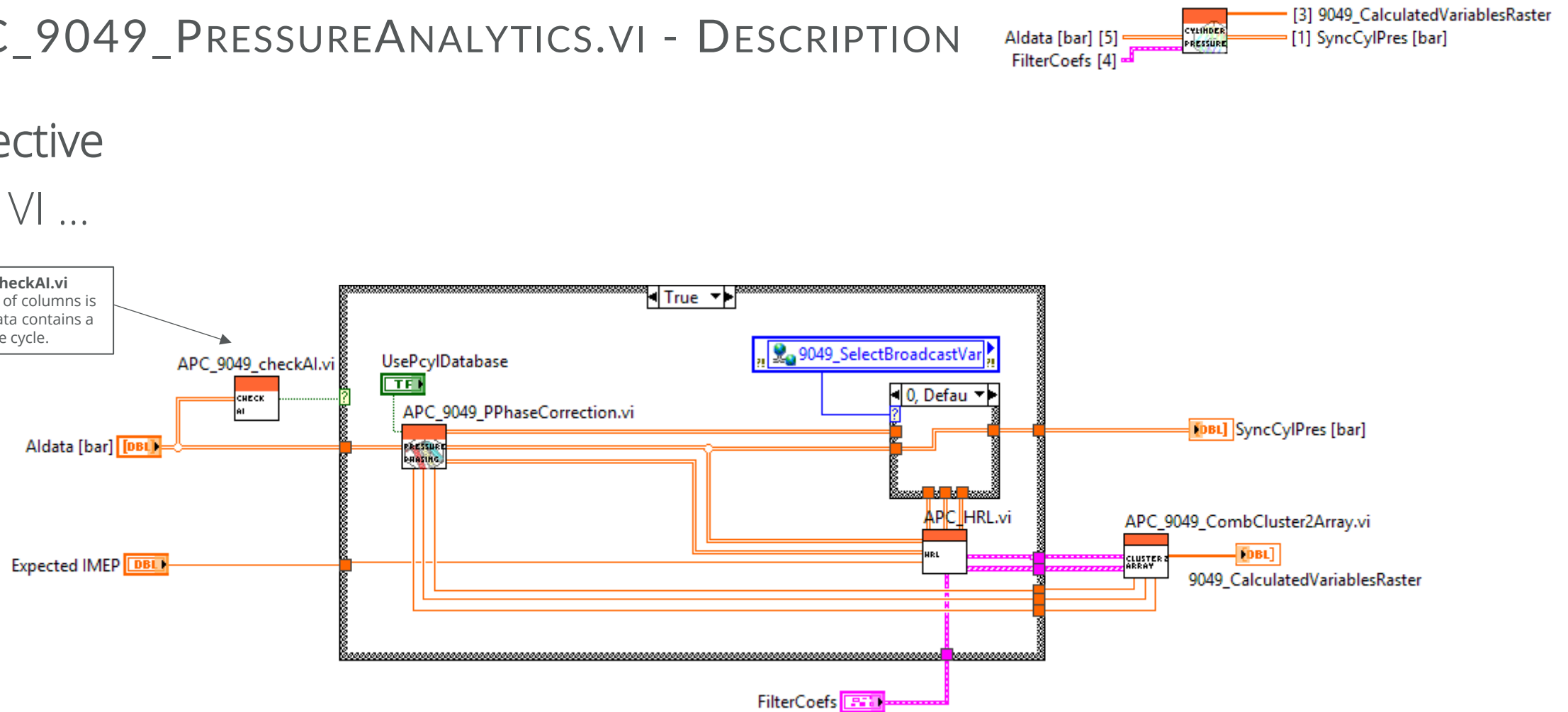
APC_9049_PressureAnalytics.vi

APC_9049_PRESSUREANALYTICS.VI - DESCRIPTION

Objective

This VI ...

APC_9049_checkAI.vi
Verify that the # of columns is 7200, i.e., the data contains a full engine cycle.



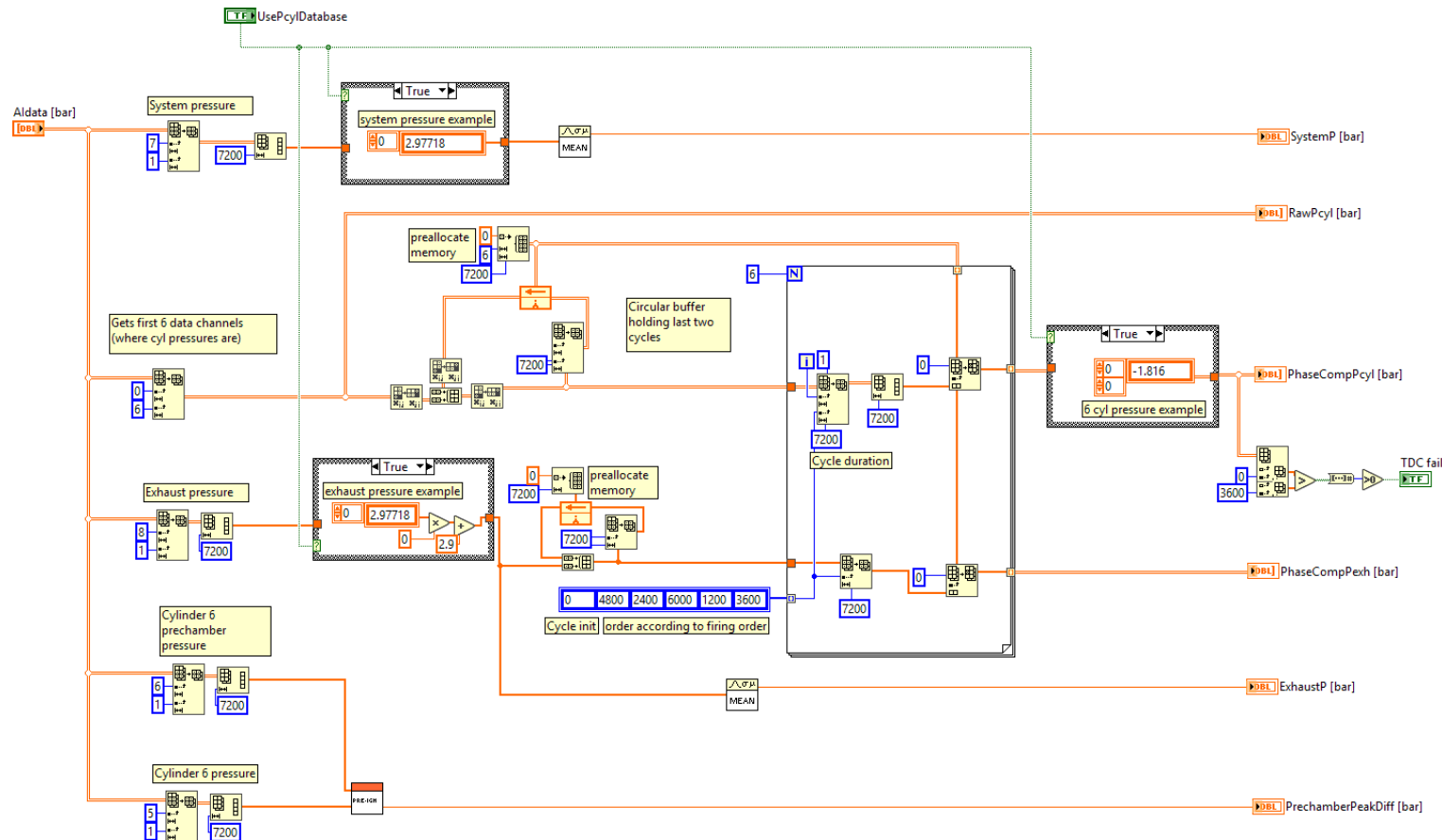
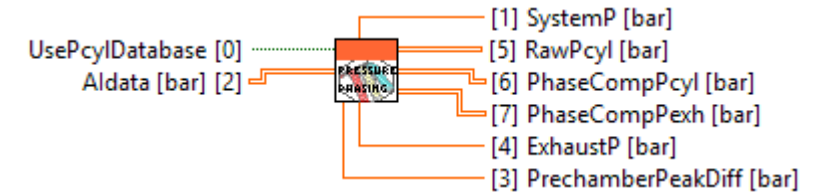
TITLE

APC_9049_PPhaseCorrection.vi

APC_9049_PPHASECORRECTION.VI - DESCRIPTION

Objective

This VI ...



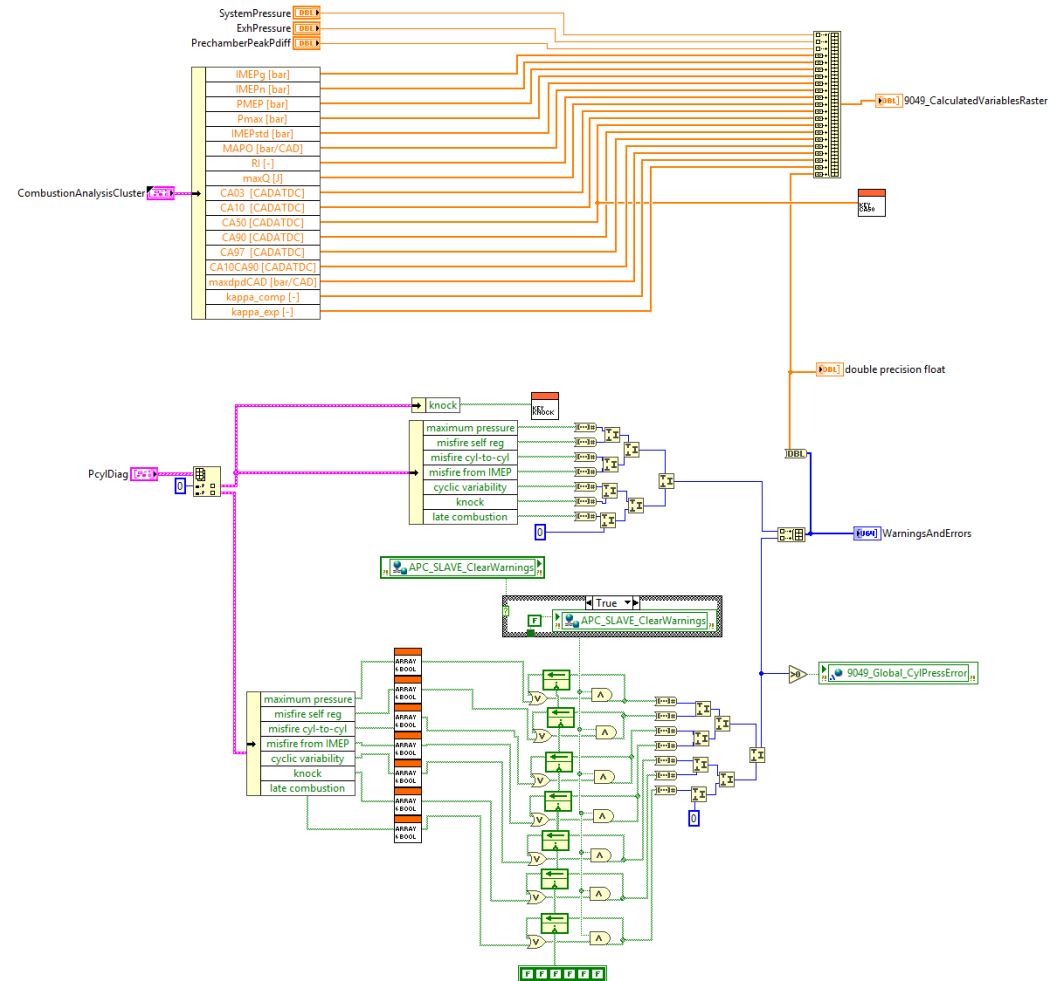
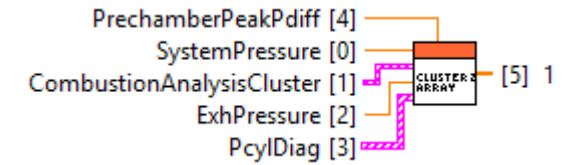
FLAG IRREGULAR COMBUSTION

APC_9049_CombCluster2Array.vi

APC_9049_COMBCLUSTER2ARRAY.VI - DESCRIPTION

Objective

This VI ...



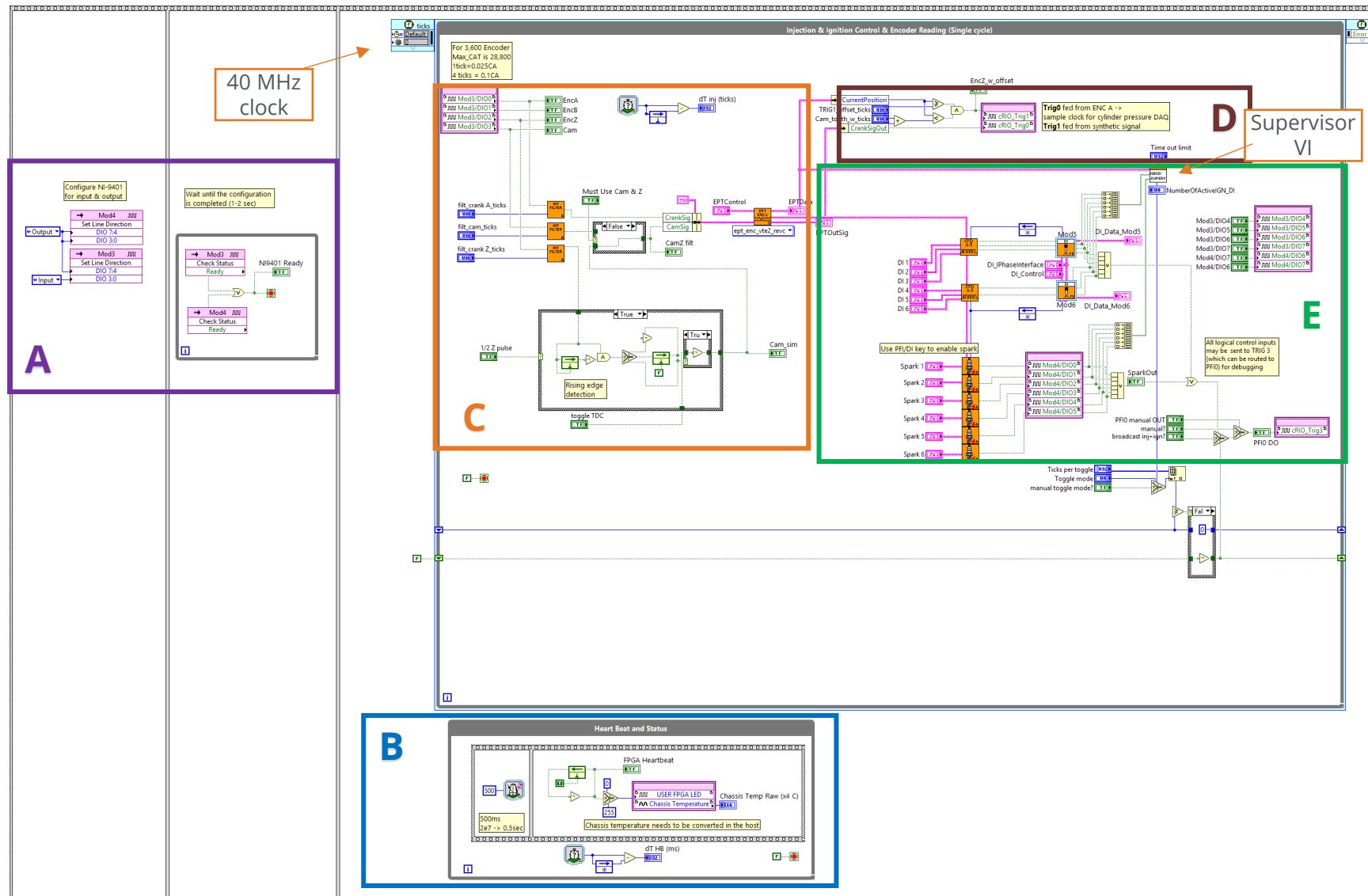
ON-FPGA MAIN VI (cRIO 9049)

APC_9049_FPGA_main.vi

cRIO 9049 FPGA MAIN VI - DESCRIPTION

Objective

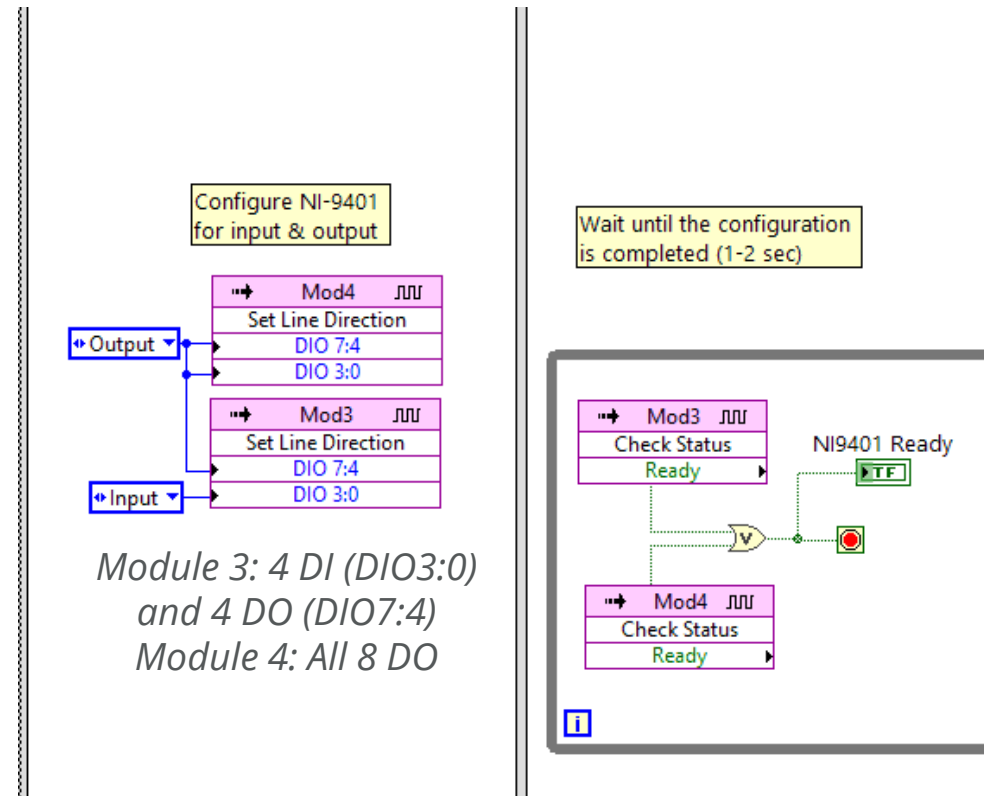
cRIO 9049 FPGA MAIN VI - OVERVIEW



cRIO 9049 FPGA MAIN VI - BD DETAILS 1/X

A

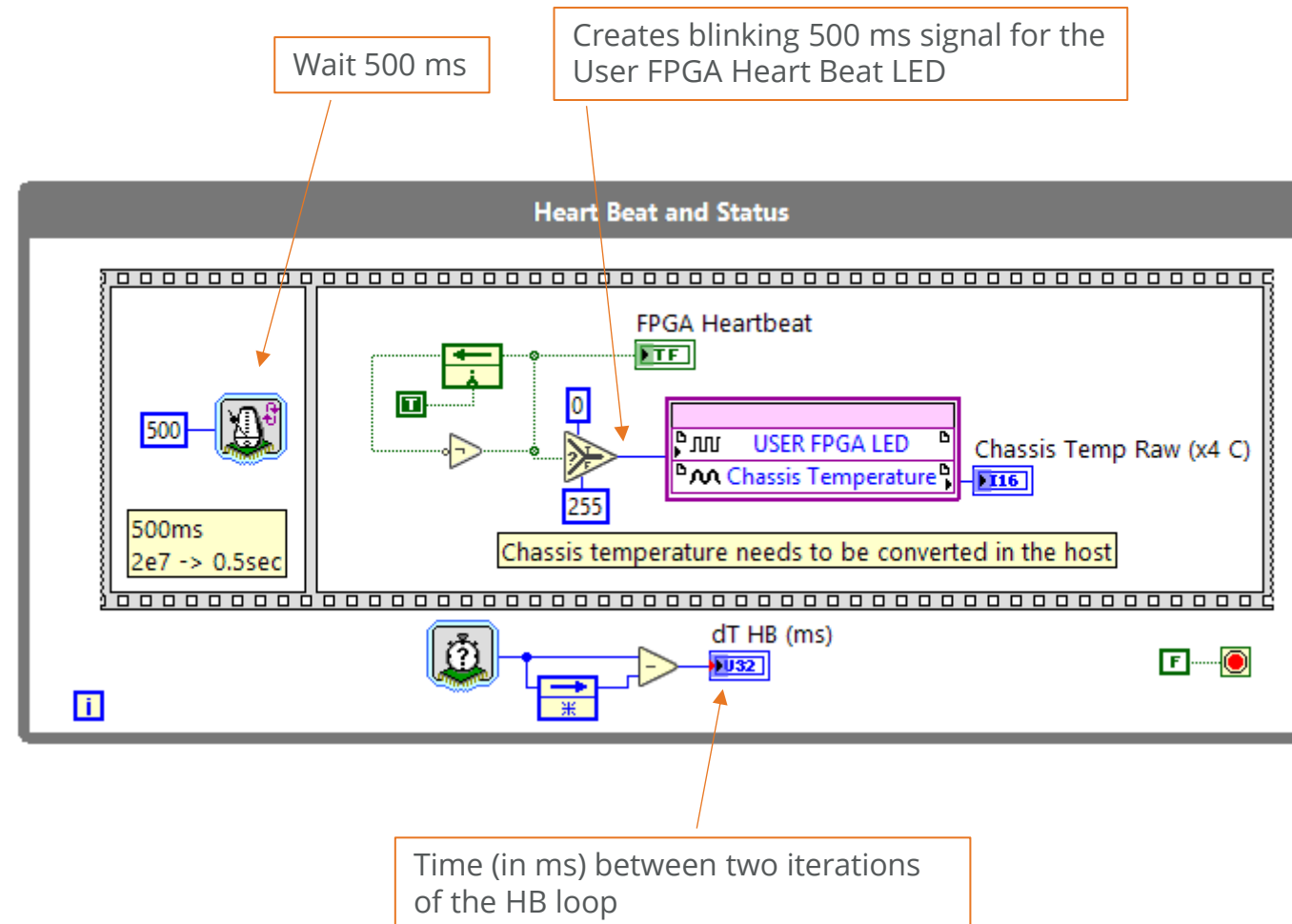
Section **A**: Configure NI-9401 I/O and wait for the configuration to be completed before executing the main FPGA logic



cRIO 9049 FPGA MAIN VI - BD DETAILS 2/X

B

Section B: Update Heartbeat status

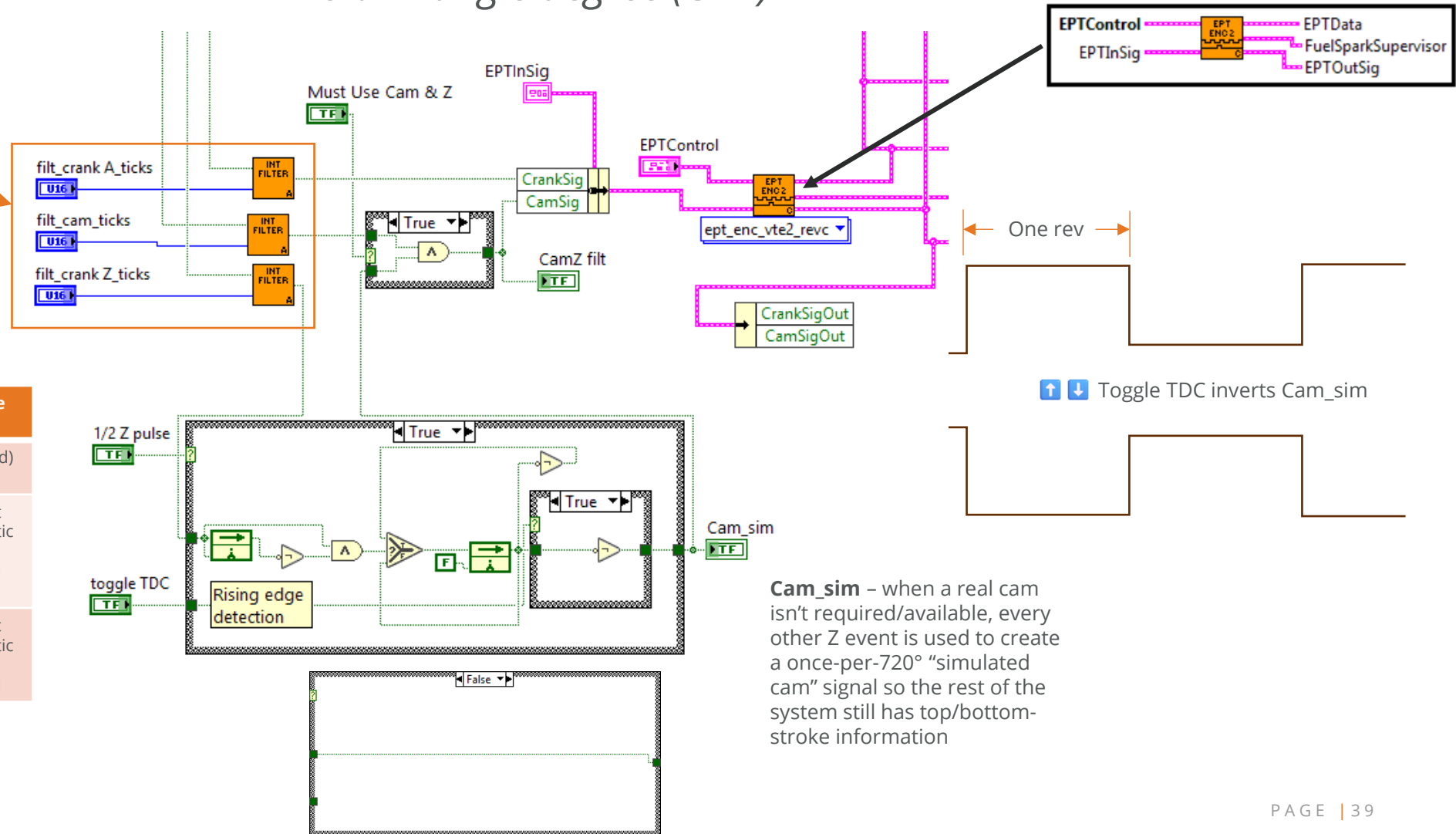


cRIO 9049 FPGA MAIN VI - BD DETAILS 3/X



Section C: Clean the raw encoder signals and derive crank angle degree (CAD)

These are deglitchers (low-pass filters) with thresholds expressed in clock ticks. They output 'clean' versions of EncA, Cam, and EncZ.



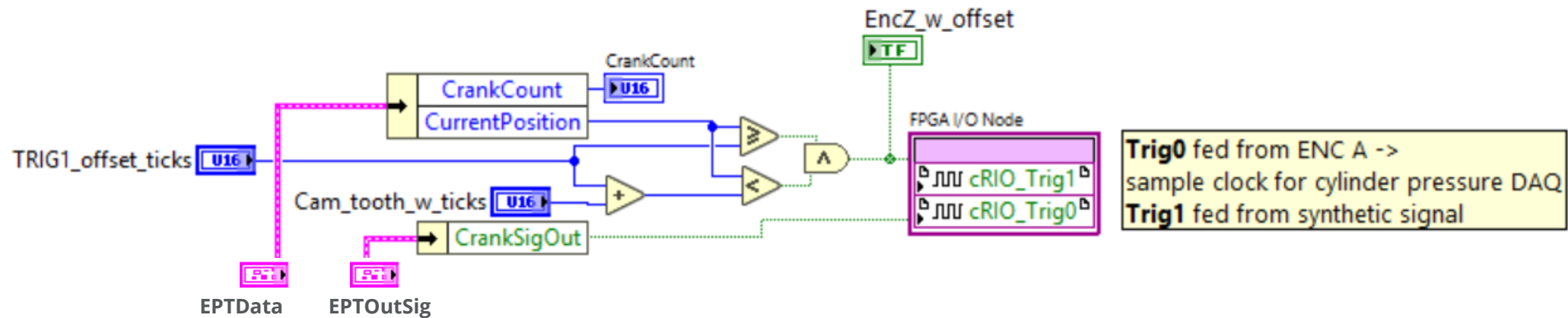
Engine / Setup	Must Use Cam & Z	1/2 Z pulse	Toggle TDC
4-stroke with real cam signal	True	(ignored)	(ignored)
4-stroke, cam-less	False	True (make 720° synthetic cam)	Invert synthetic cam signal
2-stroke (no cam signal)	False	False (360° synthetic cam)	Invert synthetic cam signal

Cam_sim – when a real cam isn't required/available, every other Z event is used to create a once-per-720° "simulated cam" signal so the rest of the system still has top/bottom-stroke information

cRIO 9049 FPGA MAIN VI - BD DETAILS 4/X



Section **D**:



Cam_tooth_w_ticks = width of one-cam-tooth in ticks
EncZ_w_offset =

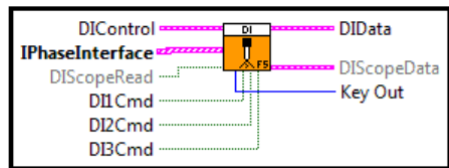
IF CurrentPosition $\geq$ TRIG1_offsetTicks
AND CurrentPosition $<$ TRIG1_offsetTicks + Cam_tooth_w_ticks
THEN EncZ_w_offset = TRUE

cRIO 9049 FPGA MAIN VI - BD DETAILS 4/X

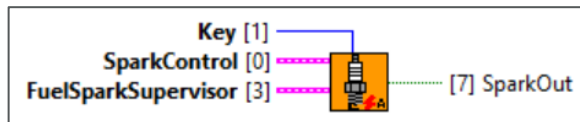


Section **E**: Command the fuel injectors and the spark plugs

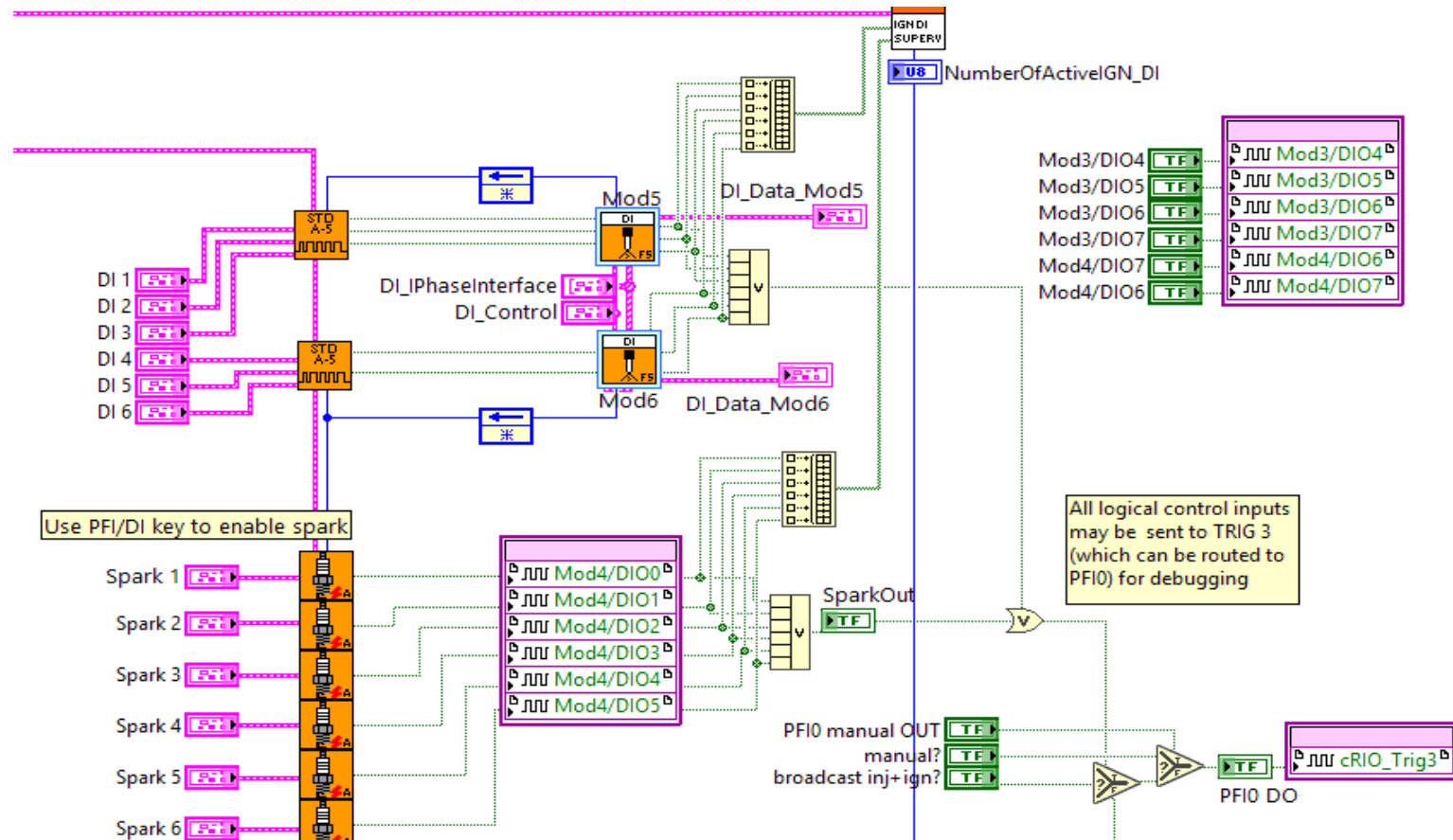
di_supv_exp_revf5.vi



esttl_vt_spark_revf5.vi



Key: Set by the *di_supv_exp_revf5.vi* to unlock spark ignition.



cRIO 9049 ON-FPGA SUPERVISOR VI

APC_9049_FPGA_IGNDI_supervisor.vi

ON-FPGA EPT SUPERVISOR VI – DESCRIPTION

Objective

Count how many IGN (spark) and DI channels are currently active, and report the total as **NumberOfActiveIGN_DI**.

Method

For each channel, maintain a timer that resets once a rise in signal is detected (change from 0 to 1).

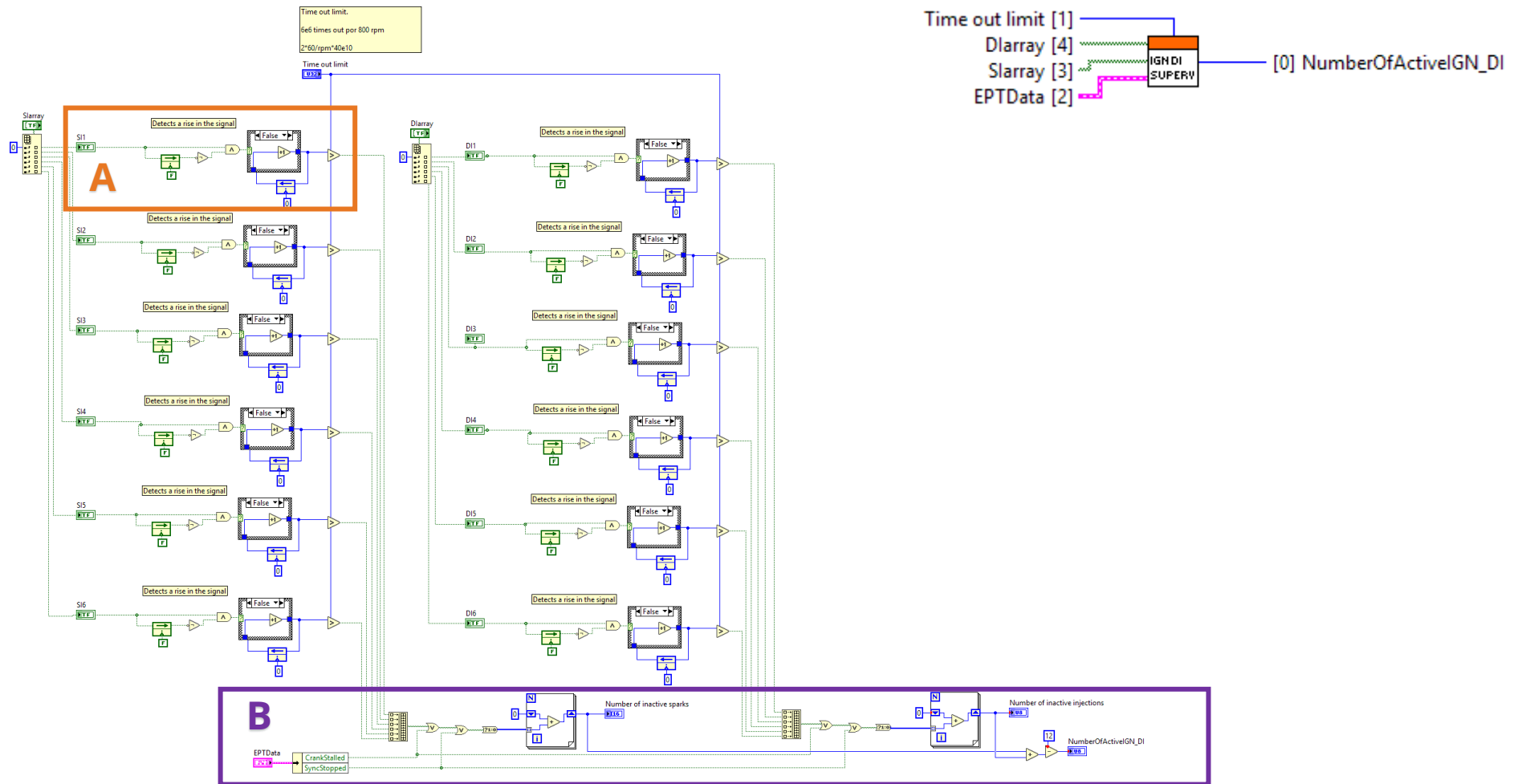
If timer exceeds the **Time out limit**, the channel is marked inactive.

Sum the active channels to get NumberOfActiveIGN_DI.

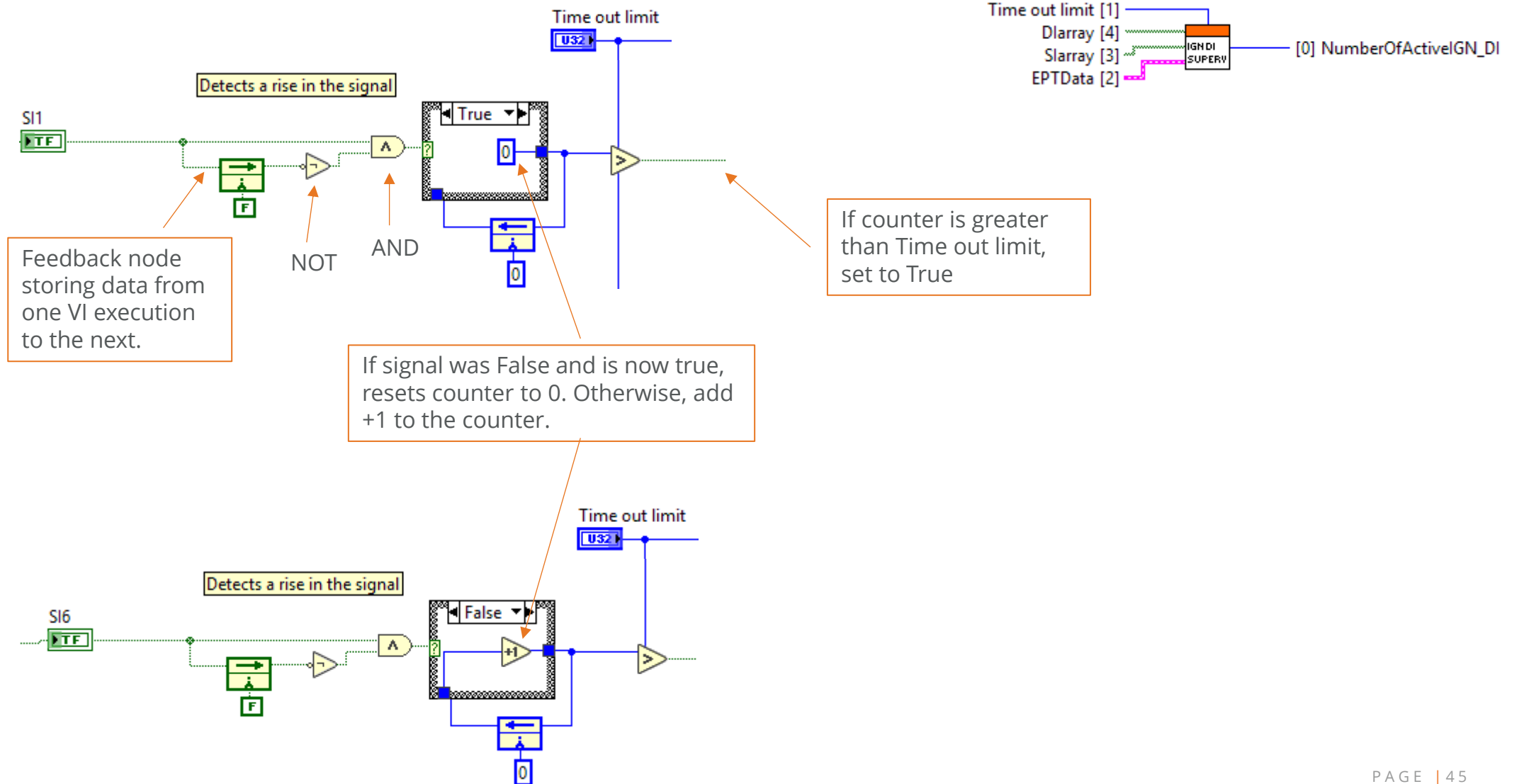
Output Example

On a 6-cylinder engine with both ignition and direct injection firing normally, it is expected that $\text{NumberOfActiveIGN_DI} = 12$ (6 IGN + 6 DI).

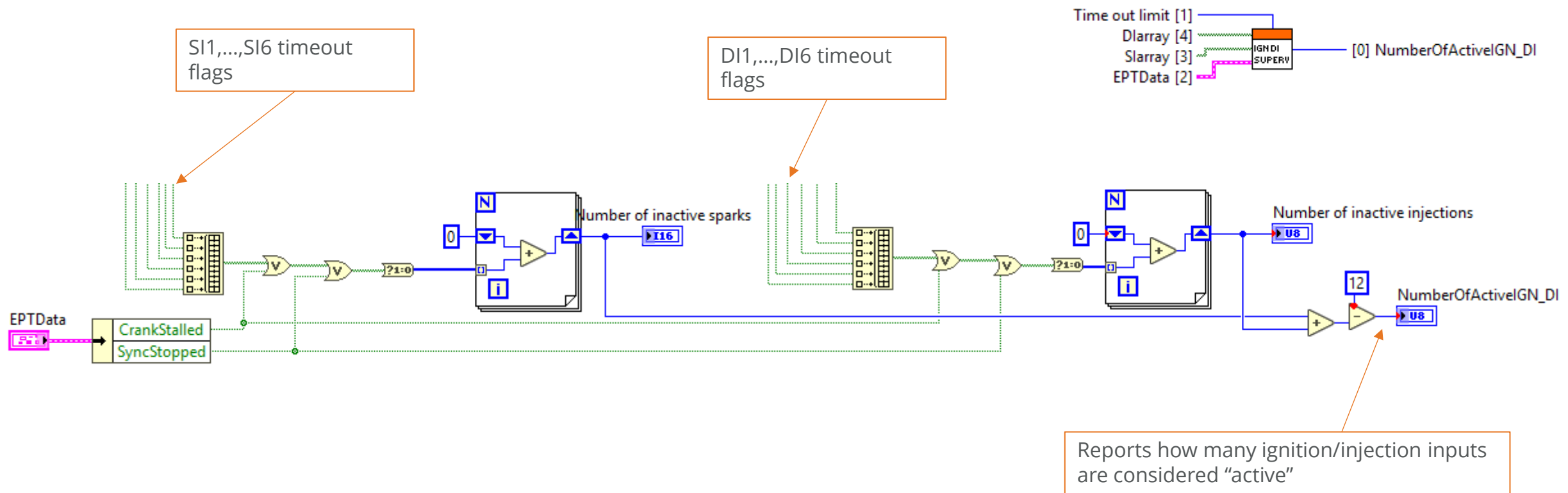
ON-FPGA EPT SUPERVISOR VI – BD OVERVIEW



ON-FPGA EPT SUPERVISOR VI – BD DESCRIPTION 1/2



ON-FPGA EPT SUPERVISOR VI – BD DESCRIPTION 2/2



RT Resources: Misc. RT-side services (e.g., variables, network streams, timing, etc.) that the RT app can use.

Chassis I/O (FPGA-accessible built-ins).

Chassis Temperature: Onboard temperature sensor reading from the controller/chassis.

Sleep: Boolean you can assert from FPGA to put the controller into low-power sleep.

System Reset: Hardware reset line for the system. Use with caution; it reinitializes the device.

USER FPGA LED: The user LED on the controller front panel, writeable from FPGA.

USER Push Button: The user button on the controller front panel, readable from FPGA.

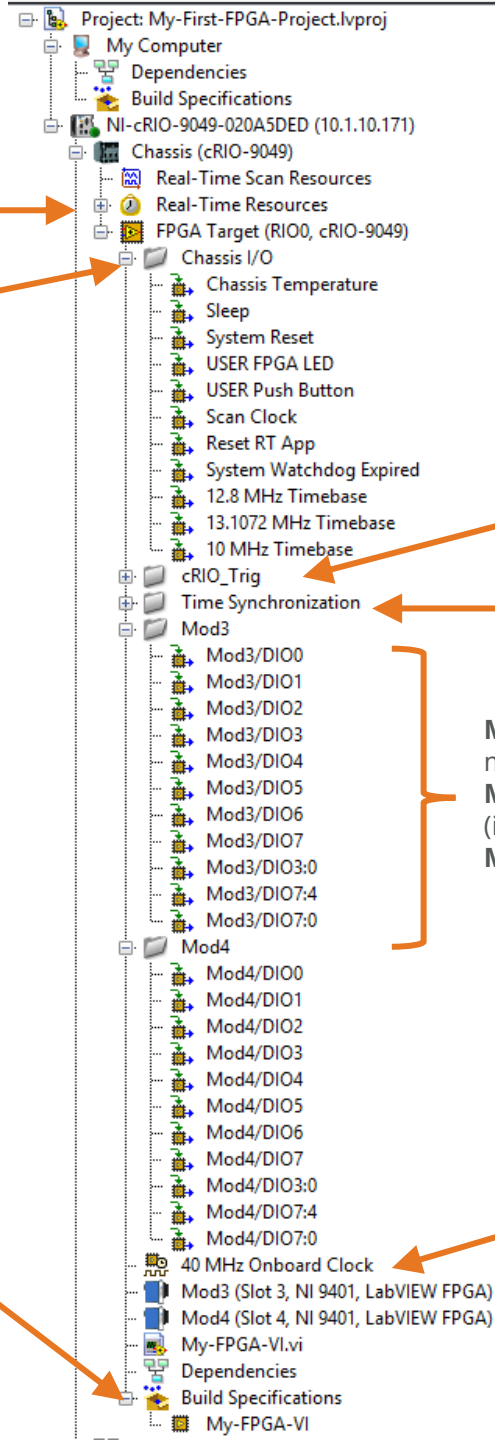
Scan Clock: A tick from the Scan Engine each scan period. Use to time/synchronize FPGA logic with Scan Engine activity.

Reset RT App: Line used to programmatically reset/restart the RT application from the FPGA (when your RT app is written to honor it).

System Watchdog Expired: Status indicating the RT watchdog has expired (e.g., RT app stopped feeding it).

12.8 MHz / 13.1072 MHz / 10 MHz Timebase: Hardware timebase clocks you can route/use to derive FPGA clocks or time measurements.

Build Specifications: FPGA build configurations (bitfile builds). You compile these to generate the .lvbitx your RT/host app loads.



cRIO_Trig: Chassis trigger bus resource for precise timing/handshaking across supported modules or external trigger lines.

Time Synchronization: FPGA hooks for system time sync infrastructure

Mod3 (Slot 3, NI 9401, LabVIEW FPGA): NI 9401 high-speed digital I/O module configured for FPGA mode.

Mod3/DIO0 .. Mod3/DIO7: Individual digital lines (Boolean). Direction (in/out) is set per nibble on the 9401.

Mod3/DIO3:0 and Mod3/DIO7:4 : 4-bit port views

40 MHz Onboard Clock: Default FPGA base clock provided by the device. Often used to drive Single-Cycle Timed Loops or derive custom clocks.

ENCODER

Senesata H25D-SS-3600-ABZC-28/5-SM18: 3,600 cycles per turn (0.1 degree resolution). See nomenclature full meaning [here](#)

CAT: Crank Angle Ticks.

EXTRAP: EPT extrapolation factor (power-of-two ticks between teeth).

EXTRAP=4

CurrentPosition: Absolute crankshaft position in CAT

MAX_CAT: Total number of CAT per engine cycle.



ept_enc_vte2_revc

CrankStallPeriod: The maximum crank tooth period between evenly spaced crank teeth in terms of 40 MHz clock ticks. A measured crank tooth period greater than CrankStallPeriod causes the Period output of the EPT VI to report as zero and the CrankStalled output to report as TRUE. Engine speed in RPM can be converted to CrankStallPeriod in 40 MHz clock ticks according to the following equation:

$$\text{CrankStallPeriod}(\text{clock ticks}) = (60 * 4E7) / (\text{EngineSpeed}(\text{RPM}) * \text{NumberOfCrankTeeth})$$

ENGINE POSITION TRACKING (EPT)

EPTData: Cluster of status information to engine position tracking.

- **CrankSigOS**: A 40 MHz one-clock one-shot generated at the rising edge of each received or simulated **CrankSig** input. This signal can be used in conjunction with **CrankCount** for any purpose within LabVIEW FPGA logic to perform engine synchronous tasks.
- **CrankCount**: Latest tooth count referenced to **crank tooth 0**, reported **over a complete engine cycle** and **as if there were no missing/extra teeth**. With a pattern like **N-M**, if **NumberOfCrankTeeth = 60** and **Stroke = TRUE** (4-stroke), **CrankCount** spans **0...119**. Use this to index tooth-based events consistently across cycles, regardless of missing-tooth patterns.

CONTROL SOFTWARE STRUCTURE

Main flow of command
→

